CR-SHE: Oblivious homomorphic aggregation for collusion-resilient search and analytics over encrypted multi-cloud IoT data

ARTICLE INFO

Keywords:
Cloud computing
Distributed point function
Function secret sharing
Homomorphic encryption
Internet of Things
Oblivious analytics
Searchable encryption

ABSTRACT

Internet of Things deployments replicate telemetry across independent clouds for availability, and the data must stay encrypted once it leaves the device domain. Distributed-trust systems already hide which records a query selects, but they operate on plaintext, stop at retrieval, or need server-to-server interaction. We present CR-SHE, in which retrieval and analytics are both oblivious to a coalition of up to $t = n-1$ colluding providers, over data encrypted under one owner's key, with no communication between providers. We show that a homomorphic layer is then necessary: an additively shared store cannot answer such queries locally. In CR-SHE each provider folds its share of the selection vector into encrypted columns and returns a single ciphertext, so neither the matched set nor its size leaves the client. A precomputed-column fast path makes registered aggregates independent of the record count, a two-round flexible path serves conjunctions and ad-hoc weights by dealing fresh shares, and a one-time-pad mask removes posting-list leakage from the stored index. We prove correctness and adaptive security against $t = n - 1$ collusion and evaluate on two real IoT corpora. On 405,184 sensor records a registered aggregate costs 39 ms per provider regardless of record count, and four aggregates with their MACs return in one 132 kB ciphertext. Against one server holding the same encrypted columns, the trust split saves 1.8–2.9× in server work but over 3,000× in uplink, which is what a thin IoT client needs. A frequency attack recovering every non-temporal tag from an unmasked index recovers none from the masked one.

1. Introduction

The proliferation of connected sensors, wearables and cyber-physical systems has made the Internet of Things (IoT) one of the largest and fastest-growing sources of continuously generated data [1]. Bedside monitors, smart meters, traffic and environmental sensors and industrial controllers emit measurements continuously, and the people who need to query them, among them clinicians, grid operators, city agencies and maintenance contractors, are rarely the ones who own the devices. The natural response is to serve this data from the cloud, and modern deployments increasingly replicate it across several providers. Multi-cloud operation supplies elastic query capacity, improves availability through redundancy across independent infrastructures, and frees data owners from single-vendor lock-in [2, 3]. Yet much of this data is acutely sensitive, including physiological telemetry in connected healthcare, mobility traces in smart cities and diagnostic streams from connected vehicles, so it must be encrypted before leaving the IoT trust domain and stay encrypted in the cloud, even as multiple stakeholders need to query and analyse it.

Encrypting outsourced data with conventional schemes neutralises the reason it was sent to the cloud. A provider holding only ciphertexts can neither locate the records a query needs nor compute over them, forcing either decryption in the cloud, which reintroduces exposure, or a wholesale download to the client, which defeats the purpose of outsourcing. Searchable encryption lets a provider identify records matching an encrypted keyword [4–6], and leveled homomorphic encryption lets it evaluate bounded-depth arithmetic directly on ciphertexts [7, 8]. Both, in their single-server form, leak the access pattern, and a substantial body of leakage-abuse work converts that leakage into plaintext recovery [9–11].

Replicating data across independent providers offers an escape that single-server cryptography cannot. If the client splits its query so that no provider sees it, the access pattern stays hidden as long as one provider abstains from collusion. Splinter [12] established this model with function secret sharing, DORY [13] brought it to encrypted documents, and Waldo [14] extended it to rich predicates over time series. The model is therefore not new, and framing a "cross-cloud collusion adversary" as an unexplored threat would misstate the state of the art. What remains open is narrower and, for IoT analytics, decisive.

Selection and computation are not the same problem, and hiding the first is worthless if the second gives it back. A scheme can privately determine which records match a keyword and then, in order to aggregate over them, hand the matched index set to the providers, at which point the coalition learns the access pattern exactly, and across repeated queries the search pattern. Existing distributed-trust systems avoid this by not computing at all (DORY), by computing over data the servers can already read (Splinter, whose databases are public), or by moving to a three-party honest-majority setting with secret-shared data and server-to-server interaction (Waldo). None of them gives analytics over singly-encrypted, owner-keyed data under a $t = n - 1$

collusion bound without inter-provider communication. This paper identifies that composition leak and eliminates it under a constraint set of $t = n - 1$ collusion, no server-to-server channel, and data the providers cannot read. These constraints rule out the obvious alternative of secret-sharing the data, because a shared store cannot answer a hidden query without either server interaction or a single-server private-retrieval primitive (Section 6.2).

We propose CR-SHE (Collusion-Resilient Searchable Homomorphic Encryption). The data owner replicates a one-time-pad-masked pseudonymous index and a homomorphically encrypted record column to n providers. A query is one distributed point function (DPF) key per provider. Rather than returning the selection vector so the client can name the matched records, each provider contracts its *own share* of that vector against the encrypted column and returns a single ciphertext; the client adds the n ciphertexts and decrypts once. The matched set is never assembled at any provider, its cardinality never leaves the client, and per-query downlink for an aggregate is one ciphertext rather than linear in the record count. The contributions are as follows.

- **End-to-end obliviousness under $t = n - 1$ collusion, and why it needs a homomorphic layer.** We give a construction in which the analytics phase inherits the access-pattern hiding of the selection phase, over data encrypted at rest under a single owner's key, with no server-to-server communication. We show that secret-sharing the data instead does not work: local answers over additive shares miss the cross terms, and any scheme that avoids them yields single-server private information retrieval (Proposition 1, Section 6.2). Section 6.3 presents the mechanism, and Remark 3 explains why returning the matched set to the providers admits no simulator, while dealing it as fresh shares does.
- **Oblivious homomorphic aggregation, and a precomputed-column fast path.** We make explicit the compatibility condition (Lemma 1) under which DPF shares and the homomorphic plaintext ring compose without wraparound. The condition is elementary in isolation, but its violation is silent and data-dependent in practice. We then give a variant (Section 6.5) in which single-keyword aggregates over pre-registered templates cost a symmetric full-domain pass plus $O(N/\ell)$ packed ciphertext operations, independent of the record count, and return the value and its MAC for several templates in the two slot rows of one ciphertext (Section 6.6).
- **A flexible path for conjunctions and ad-hoc weights.** Two rounds: a fixed-size retrieval, after which the client forms any selection vector it likes and deals it as fresh additive shares, which each provider contracts against the encrypted record column (Section 6.9). A coalition sees only uniform shares, so the guarantee is unchanged.
- **Index masking compatible with oblivious retrieval.** A one-time-pad mask, sampled exactly uniformly over $\mathbb{Z}_p$, makes the stored index perfectly hiding under the PRF assumption; the client strips the mask row after retrieval, so no provider ever sees an unmasked row or a mask value (Section 6.4, Theorem 1).
- **Detection of inconsistent providers, an authorisation requirement, and streaming ingest.** A MAC'd ciphertext column lets the client reject a response inconsistent with its tag except with probability $1/p \approx 2^{-36}$, which is fault detection rather than integrity. With a single MAC key a consistent linear substitution passes; per-row keys on the fast path catch it unless it touches only the queried row, and we measure both. We show that providers cannot restrict which rows a DPF-hidden query reads, measure the index extraction this permits, and identify the key-carried authorisation proof it requires. An append-only two-tier index supports IoT streams, with forward and backward privacy immediate because the coalition's per-query view is query-independent.
- **Evaluation** on two real IoT corpora, one of environmental sensor telemetry and one a twelve-site air-quality corpus with three times the tag domain, against ORAM, single-server searchable encryption, distributed-trust search, a faithful homomorphic equality scan, and a single server holding the same encrypted columns, which isolates what the non-collusion assumption buys. An empirical leakage-abuse attack quantifies what index masking removes. Succinct keys are evaluated for $n = 2$; for $n \in \{3, 4\}$ the prototype uses a non-succinct instantiation that shows provider compute is flat in n. An open reference implementation reproduces the construction and the new experiments (Section 9).

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 fixes notation and primitives. Sections 4 and 5 present the system and threat models. Section 6 details the construction, Section 7 the security analysis, and Section 8 the asymptotic costs. Section 9 describes the implementation, Section 10 reports and discusses results, and Section 11 concludes.

2. Related work

2.1. Distributed-trust query systems

The line of work closest to ours splits a query across replicated servers so that no server sees it. Splinter [12] established the model: the client splits a query into function-secret-sharing shares, each provider evaluates its share against its replica, and the query stays hidden as long as one provider does not collude. Splinter supports SUM, COUNT, MAX and TOPK, and pads result sets to a public bound. Its databases, however, are *public*, so the providers hold plaintext, confidentiality of the data at rest is out of scope, and the aggregates are computed over records the providers can already read.

DORY [13] moves to encrypted data. It masks each bit of a Bloom-filter search index so that an attacker compromising every trust domain does not immediately recover the index, hides search access patterns and the number of matching documents from an attacker controlling all but one server, and detects servers that deviate. DORY stops at retrieval: there is no computation over the matched documents, which is precisely the step at which a naive extension leaks.

Waldo [14] adds rich functionality for time-series data, supporting multi-predicate filtering while protecting contents, filter values and access patterns, with malicious security. It does so in a three-party honest-majority setting with secret-shared data and server-to-server interaction, that is, a collusion bound of $t = 1$ rather than $t = n - 1$, and without a single-owner key under which the data remains encrypted at rest. CR-SHE trades Waldo's predicate expressiveness and full malicious security for a stronger collusion bound, no inter-provider communication, and owner-keyed encryption at rest; Table 1 makes the comparison precise.

Adjacent systems built on function secret sharing target different problems: metadata-hiding communication [15], private aggregate statistics that stay robust to malformed client submissions [16, 17], and private access control for FSS dealers [18], which is the kind of mechanism Section 6.8 shows CR-SHE's authorisation requirement calls for. The underlying primitives are due to Gilboa and Ishai [19] and Boyle, Gilboa and Ishai [20, 21], and verifiable DPFs [22] let servers check that a key encodes a point function. At bottom, DPF-based selection is multi-server private information retrieval [23]: our fast path (Section 6.5) is exactly a PIR query into an owner-encrypted table of precomputed aggregates, and its linear server work is the price that PIR without preprocessing must pay [24]. What CR-SHE adds to that primitive is the ciphertext-side contraction that keeps analytics oblivious on both paths, and the mask that makes a replicated index safe at rest. More recent FSS systems push throughput [25] or remove the trusted dealer [26]; both are orthogonal to the composition studied here.

2.2. Private information retrieval

PIR with preprocessing breaks the linear lower bound: offline hints let the online phase run in sublinear time, with two servers [27] or one [28, 29]. Those hints are client state computed over the whole database and invalidated by updates, which suits a stable table read by one client better than a stream ingested continuously and queried by many stateless data users; we keep the linear pass and list preprocessing as future work. Single-server PIR from lattice-based encryption [30–33] removes the non-collusion assumption altogether [34]. For the fast path in isolation it is a genuine alternative, being a PIR query into a table of owner-encrypted aggregates, and we measure it directly as baseline (vi) (Section 6.6): on that path the trust split saves a constant factor in server work and three to four orders of magnitude in uplink, not asymptotic work. On retrieval and the flexible path a single server must multiply an encrypted selection vector by the whole $N \times N_d$ index, at least NN_d/ℓ homomorphic products, which we bound at no less than three times CR-SHE's retrieval in the same implementation; again a constant factor, with the uplink gap on top.

2.3. Encrypted search and its leakage

Symmetric searchable encryption gives sublinear search at the cost of search- and access-pattern leakage, which leakage-abuse attacks convert into query and plaintext recovery given modest auxiliary information [9–11]. The standard mitigations are padding, ORAM [35], volume-hiding structured encryption [36, 37], or distributed trust. Volume-hiding schemes are the direct counterpart of our index mask: they hide response volumes from a single server, whereas the mask hides the whole membership index at rest from every provider and leaves query-time hiding to the DPF. Multi-cloud SSE variants [38, 39] reduce leakage but scale their hiding overhead with the number of clouds and keep the index within a single trust domain; other work broadens expressiveness or adds committee-based authorisation [40–42]. Rather than asserting that our residual leakage is benign, Section 10.9 reproduces a frequency-based attack against the unmasked variant of our own index and measures what masking removes.

2.4. Computation over encrypted IoT data

Leveled homomorphic encryption [7, 8] supports bounded-depth arithmetic on ciphertexts and underpins encrypted analytics for time-series and industrial IoT workloads [43, 44]. Encrypted time-series systems such as Timecrypt [45] and Zeph [46] provide efficient aggregation but filter only on time and reveal the queried interval to the server. Purely homomorphic keyword search is possible in principle, but a homomorphic equality scan costs many ciphertext operations

Table 1
CR-SHE against distributed-trust query systems and encrypted-search baselines. *Oblivious analytics* means the servers learn neither which records feed the aggregate nor how many.

System	Storage at the servers	Search / filter	Oblivious analytics	Collusion bound	Server channel	Malicious security	Hides $\|S\|$
Splinter [12]	plaintext replica	✓	○ (plaintext)	$n-1$	no	×	pad to k
DORY [13]	encrypted replica	✓	×	$n-1$	no	detect	✓
Waldo [14]	secret-shared	✓	✓	1 (h. maj.)	yes	✓	✓
Timecrypt, Zeph [45, 46]	encrypted, 1 server	time only	✓	n/a	n/a	×	×
Multi-cloud SSE [38]	encrypted	✓	×	×	no	×	×
Pattern-hiding SSE [39]	encrypted	✓	×	○	no	×	○
SHAMC [52]	encrypted	○	✓	×	yes	×	×
(n,n)-shared, local (§6.2)	secret-shared	✓	×‡	$n-1$	no	n/a	n/a
Single server, enc. selection	encrypted, 1 server	✓	✓ (fast)	none needed	n/a	×	✓
CR-SHE (this work)	encrypted replica	✓	✓	$n-1$	no	detect	✓ / pad†

✓: supported; ○: partial; ×: not supported. *Detect* denotes detection of a deviating server without guaranteed output delivery. †Aggregates hide $|S|$; retrieval pads to the public bound $k_{\max}$. ‡Local answers over additive shares miss the cross terms (Proposition 1); fixing that needs server interaction or a single-server PIR per share. The single-server row is baseline (vi): no non-collusion assumption, but an encrypted query linear in N and no oblivious retrieval or flexible path at comparable cost.

per item, which is why CR-SHE keeps symmetric-key work on the selection path and spends homomorphic operations only on the aggregate. Within IoT, privacy-preserving search has been studied for sensing platforms [47] and vehicular networks [48], and multi-party computation with homomorphic encryption addresses the multi-owner case [49].

2.5. Multi-cloud storage

Hybrid cryptosystems, secret sharing and attribute-based access control disperse trust across providers [50, 51], SHAMC combines secure computation with partial homomorphic encryption for cross-cloud database operations [52], and systematisations of multi-cloud cryptography note the absence of standardised trust-model definitions [53]. These designs largely treat providers as independent storage and do not process queries over the ciphertexts. Encrypted and oblivious database systems [54–56] do process queries, but rely on a trusted proxy or trusted hardware in front of the store rather than on non-colluding providers.

3. Preliminaries

This section fixes notation (Table 2) and recalls the primitives CR-SHE combines: a pseudorandom function for the index mask, additive secret sharing, distributed point functions for private selection across the replicated index, and leveled homomorphic encryption for aggregation on ciphertexts.

Pseudorandom functions. $F : \{0,1\}^\lambda \times \{0,1\}^* \to \{0,1\}^\lambda$ is a PRF if for every PPT distinguisher $\mathcal{B}$, $\mathbf{Adv}^{\mathrm{prf}}_{F,\mathcal{B}}(\lambda) = |\Pr[\mathcal{B}^{F_k(\cdot)} = 1] - \Pr[\mathcal{B}^{R(\cdot)} = 1]| \le \mathsf{negl}(\lambda)$ for a uniformly random key k and a truly random function R [57].

Additive secret sharing. An (n,n) additive sharing splits $m \in \mathbb{Z}_p$ into $(m_1,\dots,m_n)$ with $\sum_j m_j = m$; any $n-1$ shares are uniform and independent of m.

Distributed point functions. An n-party DPF [19–21] for point functions $f_{\alpha,\beta} : [N] \to \mathbb{G}$ is a pair $(\mathsf{Gen}, \mathsf{Eval})$ with *correctness* $\sum_{j\in[n]} \mathsf{Eval}(j, \kappa_j, x) = f_{\alpha,\beta}(x)$ for all x, and $(n-1)$-*privacy*: a PPT simulator $\mathcal{S}_{\mathrm{DPF}}$ exists such that for every $C \subseteq [n]$ with $|C| \le n-1$, $\{\kappa_j\}_{j\in C} \approx_c \mathcal{S}_{\mathrm{DPF}}(1^\lambda, C, N)$. The two-party instantiation has keys of size $O(\lambda \log N)$ and admits full-domain evaluation in $O(N)$ pseudorandom-generator calls through a single traversal of the GGM tree, rather than the $O(N \log N)$ that N independent Eval calls would cost. For $n > 2$, succinct $(n-1)$-private constructions exist whose key size grows with both n and $\sqrt{N}$ [20]. Our prototype does not implement one: it instantiates $n > 2$ with a simpler sharing in which $n-1$ providers hold a λ-bit seed whose counter-mode expansion is their share, and one provider holds the explicit difference vector. That construction is correct by inspection and $(n-1)$-private under the PRF assumption: any coalition of $n-1$ providers is missing at least one seed expansion, which would hide α perfectly were it truly random, and is a PRF output in fact. Its per-provider key is λ bits for $n-1$ providers and $N\lceil \log p \rceil$ bits for one. This is not function secret sharing in the succinct sense, being the additive sharing of a unit vector familiar from multi-server PIR, so our key-size claims are for $n = 2$, and we use $n \in \{3,4\}$ only to show that provider compute is flat in n; the uplink it costs is reported separately (Section 10.8).

Leveled homomorphic encryption. $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ is correct for arithmetic circuits of multiplicative depth $\le L_{\mathrm{HE}}$ over plaintext ring $\mathbb{Z}_p$ and IND-CPA secure with advantage $\mathbf{Adv}^{\mathrm{cpa}}$ [7, 8]. We write $\boxplus, \boxminus$ for ciphertext addition and subtraction and $\boxdot$ for plaintext-to-ciphertext multiplication, and use SIMD packing with ℓ slots. Providers hold only pk; sk never

Table 2
Notation.

Symbol	Meaning
λ	security parameter
PPT, negl	probabilistic poly. time; negligible function
$\approx_c$	computational indistinguishability
$n,\ t$	number of providers; collusion bound ($t = n - 1$)
$N,\ N_d$	index (tag) domain size; number of records
p	HE plaintext modulus; also the DPF output group
ℓ	number of SIMD slots per ciphertext
Q	number of adaptive queries
D	secret, append-only tag dictionary, $w \mapsto \alpha_w$
$F',\ K'$	PRF for the index mask; its key, fresh per static tier
$I,\ I'$	membership index in $\{0,1\}^{N \times N_d}$; masked index
P	one-time-pad mask, rejection-sampled from $F'_{K'}$, uniform on $\mathbb{Z}_p$
α_w	row of keyword w, $\alpha_w = D(w)$
b	record selection vector, $b = I[\alpha_w] \in \{0,1\}^{N_d}$
b_j	provider CP_j's additive share of $b + P[\alpha_w]$
(Gen, Eval)	distributed point function (DPF)
κ_j	DPF key sent to provider CP_j
e_j	full-domain evaluation vector, $e_j[x] = \mathsf{Eval}(j, \kappa_j, x)$
$\mathcal{S}_{\mathrm{DPF}}$	DPF simulator
$pk,\ sk$	HE public / secret key
L_{HE}	HE multiplicative-depth bound
$m_i,\ \mathsf{CT}_i$	computable field of record i; its ciphertext
$\delta,\ \widehat{\mathsf{CT}}_i$	MAC scalar; MAC column $\mathsf{Enc}(\delta m_i)$, in slot row 1
$\delta_x,\ K''$	per-row MAC key $F''_{K''}(x)$ (fast path); its key
$f,\ w_f$	aggregate template; its public weight vector
$A_f,\ \widehat{A}_f$	precomputed aggregate column; its MAC column
$v,\ v_j$	flexible selection vector over records; CP_j's share
S	matched record set; $\lvert S\rvert$ its cardinality
$\mathcal{L}$	leakage profile
C	corrupted provider coalition ($\lvert C\rvert \le t$)
DO, DU, CP_j	data owner; data user; j-th cloud provider

leaves the DO or DU. Opaque payloads use an IND-CPA symmetric scheme.

4. System model

CR-SHE spans an IoT trust domain and an untrusted multi-cloud back-end across five classes of entities (Table 3). The **Data Owner (DO)**, an IoT-domain authority such as a hospital or fleet operator, is the root of trust, holding the tag dictionary D, the mask key K', the MAC scalar δ, and the HE secret key sk. The DO also retains the plaintext records, in cold storage, since it serves no queries, because it needs them to precompute columns and rebuild tiers; outsourcing therefore buys availability and query access for many data users, not a smaller footprint at the owner. **Data Producers** and **Edge relays** lie inside the DO trust domain, generating and transporting

Table 3
Entities, trust status, and what each holds.

Entity	Role	Trust	Holds
Data owner	builds, outsources, authorises	trusted	D, K', δ, sk; plaintext
Producers	generate records	trusted	raw data
Edge relays	transport, buffering	untrusted	nothing secret
Providers	store, select, HE-evaluate	semi-honest, $\le t$ collude	I', CT, $\widehat{\mathsf{CT}}$, A_f, $\widehat{A}_f$, pk, κ_j, v_j
Data users	query, combine, decrypt	authorised	queries, D, K', delegated sk

raw records with no secret material. The n **Cloud Providers** $\mathrm{CP}_1, \ldots, \mathrm{CP}_n$ are independent and semi-honest: each holds only pk and a replicated copy of the masked index, the ciphertext columns and the precomputed columns, executing selection and homomorphic aggregation but never decrypting. Authorised **Data Users (DU)** issue queries, combine the per-provider ciphertexts, and hold delegated decryption.

The workflow (Fig. 1) has four stages. In *setup* (Algorithm 1), the DO derives keys, builds the pseudonymous membership index and masks it, encrypts the computable field of every record and its MAC counterpart, precomputes the aggregate columns for each registered template, and replicates everything to the n providers. To *query*, an authorised DU looks up the keyword's row in D and sends one DPF key per provider together with the template identifier. Each provider performs a full-domain evaluation and, without ever assembling the matched set, returns a single ciphertext. The DU *combines* the n ciphertexts and *decrypts* once. Because the per-provider keys are individually meaningless and every response is a fixed-size ciphertext, no network-visible message depends on the queried keyword, the matched set, or its size.

A concrete deployment. Consider a city environmental-monitoring network like the second corpus of Section 10.14. Sensor stations (producers) publish readings through gateways (edge relays) to the municipal data office (DO), which replicates the encrypted, masked store to two or three commercial clouds. Analysts at a health agency, a transport authority and a university (DUs) run registered templates, such as the mean temperature at a named site, the number of hours with carbon monoxide in the top decile, or a weighted exposure score, without any cloud learning which site or condition was asked about, or how many hours matched. Gateways only forward records over authenticated channels; all key material stays with the DO and the analysts.

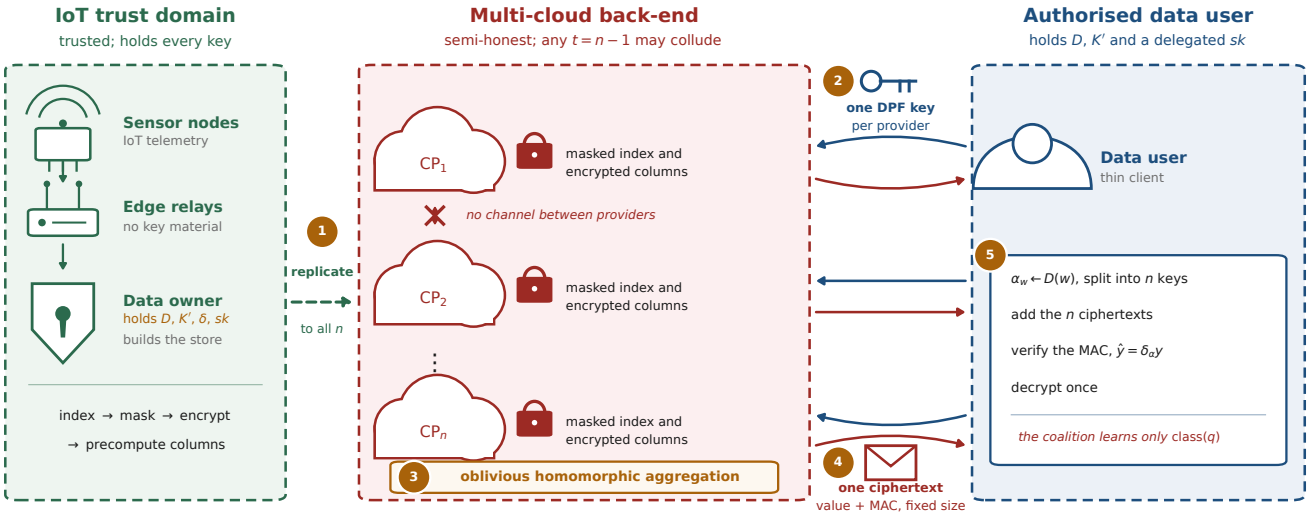


Figure 1: Overview of CR-SHE. The owner replicates a masked index and encrypted columns to n providers (1). A query is split into one DPF key per provider (2), each provider aggregates obliviously over its own share (3) and returns one ciphertext carrying value and MAC (4), and the user combines, verifies and decrypts (5). No provider ever sees the matched set or its size.

5. Threat model

The adversary $\mathcal{A}$ is probabilistic polynomial-time, static, and semi-honest: every corrupted party follows the protocol faithfully but pools and analyses everything it observes. Before execution, $\mathcal{A}$ fixes a corruption set $C \subseteq [n]$ with $|C| \leq t$; the headline guarantee is $t = n - 1$, so security holds as long as at least one provider stays out of the coalition. The defining feature is *collusion*: the corrupted providers act as a single coalition sharing state, keys, tokens and transcripts. For its chosen C, the adversary observes the full static state and complete per-query transcript of every corrupted provider,

$$\mathrm{View}_C = \big(I',\ \mathrm{CT},\ \widehat{\mathrm{CT}},\ pk,\ pp,\ \{A_f, \widehat{A}_f\}_f,\ \{\kappa_{q,j}, v_{q,j}\}_{j\in C,\, q\in[Q]}\big),$$

together with the responses those providers themselves generate. Queries are adaptive. The security goal is that this coalition view be simulatable from data-independent leakage revealing neither the keyword, the access pattern, the result-set size, nor the result.

The surrounding trust assumptions are explicit. The DO and authorised DUs are honest and hold the secret keys; data producers and in-domain edge relays hold no secret material. All channels are authenticated and confidential, so wiretapping yields nothing beyond what the corrupted providers already see, and messages cannot be tampered with.

Four boundaries delimit the model. First, providers are semi-honest for the main theorem; Section 6.7 adds detection of responses inconsistent with their MAC, but full malicious security, in particular against a provider that applies a consistent but different contraction, requires verifiable function secret sharing and is left to future work. Second, collusion between a data user and a provider is excluded, since an authorised user already holds decryption capability. We do, however, treat a client that exceeds its authority as in scope: Section 6.8 shows that providers can neither restrict which rows a hidden query reads nor reject keys for arbitrary linear combinations, which the semi-honest provider model does not cover. Third, a single compromised DU exposes all data, because decryption capability is delegated in full; threshold or attribute-based key delegation is a natural extension we do not evaluate. Fourth, side channels and traffic-volume leakage outside the message sizes accounted for here are assumed mitigated by orthogonal system-level defences.

6. The CR-SHE construction

6.1. Overview

The DO builds a membership index $I \in \{0,1\}^{N \times N_d}$ in which keyword w occupies row α_w and $I[\alpha_w][i] = 1$ if and only if record i contains w. Rows are assigned by a secret, append-only dictionary D: a keyword receives the next free row when it first appears and keeps it for the lifetime of the deployment, so rows never move when keywords are added (Section 6.10). The DO shares D with authorised DUs and keeps it from the providers, because the vocabulary itself, being device and site names, is sensitive. Which row a keyword gets carries no security weight, since providers see neither addresses nor unmasked rows; only injectivity matters, for correctness. Rows are fixed width, so no row is

distinguishable by size, and the index is stored masked as $I'[x][i] = I[x][i] + P[x][i] \bmod p$ with P drawn from the PRF $F'_{K'}$ by rejection sampling, so that every entry is uniform on $\mathbb{Z}_p$ (Section 6.4). The computable field of each record is encrypted as $\mathrm{CT}_i = \mathsf{Enc}_{pk}(m_i)$, with a MAC column $\widehat{\mathrm{CT}}_i = \mathsf{Enc}_{pk}(\delta m_i)$; opaque payloads are encrypted symmetrically.

A design that first recovers the selection vector at the client and then asks the providers to evaluate f over $\{\mathrm{CT}_i\}_{i\in S}$ destroys in the second step exactly what the first protected, because the providers are handed S. CR-SHE therefore never materialises S at a provider. Each provider contracts its own share of the selection vector against the encrypted column and returns one ciphertext.

6.2. Why a homomorphic layer is necessary

The obvious alternative to replicating ciphertexts is to deal the data to the providers as an (n, n) additive sharing over $\mathbb{Z}_p$ and answer DPF queries over the shares, with no homomorphic encryption at all. Under our constraints it does not work, and the reason fixes what any design here must contain.

Proposition 1 (No local answers over shares). *Let a column $A \in \mathbb{Z}_p^N$ be dealt as $A = \sum_{k\in[n]} S_k$, with $S_1, \ldots, S_{n-1}$ uniform and CP_k holding S_k, and let e_j be the evaluation vector of DPF key κ_j for $f_{\alpha,1}$. If each provider returns $r_j = \langle e_j, S_j\rangle$, then*

$$\textstyle\sum_j r_j - A[\alpha] \;=\; -\sum_{j\neq k}\langle e_j, S_k\rangle,$$

and for $n = 2$ this error is uniform on $\mathbb{Z}_p$ over the choice of S_1, so the answer is correct with probability $1/p$.

Proof. $A[\alpha] = \langle \sum_j e_j, \sum_k S_k\rangle = \sum_{j,k}\langle e_j, S_k\rangle$; the providers' answers supply only the diagonal. For $n = 2$, substituting $S_2 = A - S_1$ gives the error $-\langle e_1, A\rangle + \langle e_1 - e_2, S_1\rangle$, which is uniform because S_1 is uniform and independent of the keys, provided $e_1 \neq e_2$. Equality would mean $e_1 = e_2 = 2^{-1}u_\alpha$, which reveals α to either provider and so cannot occur for a private DPF. □

The cross terms are not an artefact of choosing linear answers. Take any non-interactive scheme in which CP_1 holds an additive share S_1 of the data, providers answer independently, and every query and response is shorter than a share. Given a database X held by a single server, a client can run that scheme with $S_1 := X$ and the other shares drawn by itself, which fixes the data as $A = X + \sum_{k>1} S_k$. It plays providers $2, \ldots, n$ locally, sends only the first query to the server, and subtracts $\sum_{k>1} S_k[\alpha]$ from the result. Privacy against the single provider CP_1 hides α from the server, so this is a non-trivial single-server PIR scheme for X, and such schemes imply oblivious transfer [58]; their standard instantiation is homomorphic encryption [34]. Secret-sharing the data therefore does not remove the public-key primitive, it only moves it into a per-share PIR; the alternative is to let providers interact, for example through multiplication triples, which the no-channel constraint excludes. A store replicated under one owner's homomorphic key is the direct way to supply the primitive, and it also brings three operational properties that a shared store lacks.

Durability. A sharing with privacy threshold $n - 1$ is lost when any one share is lost, whereas a replica set survives the loss of $n - 1$ providers, which is the reason multi-cloud deployments replicate at all. Threshold sharing does not escape the trade-off: a $(t{+}1, n)$ scheme tolerates $n - t - 1$ losses, so at $t = n - 1$ it degenerates to the (n, n) case.

Graceful degradation. A query needs every provider in the set it is addressed to. With replicas, queries continue over any surviving set of at least two providers, at that set's collusion bound, without touching stored state; a shared store cannot answer until the dealer re-shares.

Key management and churn. Encryption under one owner key lets the owner rotate keys, revoke a provider, or add one by copying ciphertexts. Under sharing, every change to the provider set re-deals the whole store, and the dealer must hold the plaintext to do it.

6.3. Oblivious homomorphic aggregation

Suppose the providers hold additive shares $v_1, \ldots, v_n \in \mathbb{Z}_p^M$ of a vector v and replicas of an encrypted column $\mathrm{C} = (\mathsf{Enc}_{pk}(c_1), \ldots, \mathsf{Enc}_{pk}(c_M))$. Provider j returns

$$\mathrm{ct}_j \;=\; \boxplus_{i\in[M]} v_j[i] \boxdot \mathrm{C}_i,$$

and the DU computes $\boxplus_j \mathrm{ct}_j$, which decrypts to $\langle v, c\rangle$, and decrypts once. Nothing that leaves a provider depends on v beyond its own share, and the response is one ciphertext of fixed size whatever the support of v. CR-SHE instantiates this contraction twice. On the fast path (Section 6.5) v is the DPF's unit vector u_{α_w}, its shares are the evaluation vectors e_j, and C is a precomputed aggregate column over the tag domain. On the flexible path (Section 6.9) v is a selection vector over the records that the client forms after a retrieval and deals as fresh shares, and C is the record column. In both cases the matched set is never materialised at a provider.

Lemma 1 (Algebraic compatibility). *If the DPF output group and the HE plaintext ring are the same $\mathbb{Z}_p$, and $\sum_i |w_f[i]\, m_i| < p$ for every admissible template, then for shares $b_1, \ldots, b_n \in \mathbb{Z}_p^{N_d}$ with $\sum_j b_j = \beta \bmod p$,*

$$\mathsf{Dec}_{sk}\Big(\boxplus_{j\in[n]}\ \boxplus_{i\in[N_d]}\ b_j[i] \boxdot CT_i\Big) \;=\; \sum_{i\in[N_d]} \beta[i]\, m_i \pmod p.$$

Proof. $\boxdot$ and $\boxplus$ are homomorphisms over the same ring $\mathbb{Z}_p$, so summation commutes with decryption and no reduction is lost: share addition and plaintext addition are taken modulo the same modulus. The magnitude bound prevents plaintext overflow. What remains is decryption correctness, which is a noise-budget question rather than a depth question. A plaintext-to-ciphertext product consumes no multiplicative level but scales the noise by up to the norm of the plaintext, here as large as $p \approx 2^{36}$, because DPF shares are uniform in $\mathbb{Z}_p$, and the contraction sums $\lceil N_d/\ell \rceil$ such products before a rotate-and-sum. The parameters must leave room for that growth. We check the worst case, full-magnitude coefficients across several packed chunks, empirically rather than assume it (Section 10.1). □

Lemma 1 is what makes the two primitives compose. Instantiating the DPF over $\mathbb{Z}_{2^k}$ and the homomorphic scheme over an unrelated plaintext modulus, which is the natural default since the two libraries are chosen independently, silently breaks correctness through wraparound, and the failure is data-dependent and therefore easy to miss in testing.

6.4. Masking the index

Retrieval (Section 6.8) reads a row of the membership index through the DPF. Stored in the clear, that index would hand every provider each posting-list length, which the attack of Section 10.9 turns into tag identities. The DO therefore stores it masked, $I'[x][i] = I[x][i] + P[x][i] \bmod p$, where row $P[x]$ is the output of $F'_{K'}$ on $(x, \cdot)$ turned into exactly uniform elements of $\mathbb{Z}_p$ by rejection sampling: each 64-bit PRF word is cut to $\lceil \log_2 p \rceil = 37$ bits and kept only if it is below p, which accepts a candidate with probability $p/2^{37} \approx 0.5$. A retrieval returns shares of $I'[\alpha_w] = b + P[\alpha_w]$, and the DU, which derives $P[\alpha_w]$ from K', subtracts it. No provider ever sees an unmasked row or a mask value, and no correction ciphertext is needed.

Theorem 1 (Index hiding). *For any two membership indexes $I_0, I_1 \in \{0,1\}^{N \times N_d}$, no* PPT *distinguisher tells $I_0 + P$ from $I_1 + P$ with advantage greater than $\mathbf{Adv}^{\mathrm{prf}}_{F'}$. In particular the coalition learns no posting-list length, no keyword-frequency multiset, and no membership relation from the stored index.*

Proof. Replace F' by a truly random function at the cost of $\mathbf{Adv}^{\mathrm{prf}}$. A candidate is then uniform on $\{0, \ldots, 2^{37} - 1\}$, and conditioning on acceptance makes each mask value exactly uniform on $\mathbb{Z}_p$, independently of every other. $I_b + P$ is then a one-time pad of I_b and is uniform on $\mathbb{Z}_p^{N \times N_d}$ for either b. □

Reducing 64-bit words modulo p instead, as is common, would carry a statistical term $NN_d\, 2^{-64} \approx 2^{-31.9}$ on the full telemetry index; rejection sampling removes it at the cost of about two PRF words per mask value.

Remark 1 (Stripping the mask first). An alternative contracts the masked shares b_j directly against the record column and cancels the resulting term $u_f(\alpha_w) = \sum_i w_f[i]\, P[\alpha_w][i]\, m_i$ with an encrypted correction column $U_f = (\mathsf{Enc}_{pk}(u_f(x)))_{x \in [N]}$, retrieved by the same DPF. An earlier version of our design did this. It has two defects. U_f must be precomputed by the DO for each weight vector, so it serves only registered weights, for which the fast path is cheaper; it cannot serve the ad-hoc weights and conjunctions it was meant for. And the correction must itself be encrypted: published in the clear, the count correction $g(x) = \sum_i P[x][i]$ gives $\sum_i I'[x][i] - g(x) = |I[x]|$, the row Hamming weight, restoring the volume leakage the mask removes. The flexible path avoids both by stripping the mask at the client before any contraction.

6.5. Precomputed columns: the single-keyword fast path

The contraction of Section 6.3 costs $\Theta(NN_d)$ ring operations per provider per query, which is the price of hiding the access pattern without preprocessing, by the lower bounds underlying ORAM and PIR [24, 35]. For the common case, an aggregate over the whole posting list of a single keyword under a template registered at setup, this cost is avoidable. The DO precomputes, for every $x \in [N]$,

$$A_f[x] \;=\; \sum_{i \in [N_d]} I[x][i]\, w_f[i]\, m_i \bmod p,$$

and publishes it as $\lceil N/\ell \rceil$ packed ciphertexts. A query then reduces to an oblivious retrieval of one element of an encrypted length-N array:

$$\mathrm{ct}_j \;=\; \boxplus_{x \in [N]} e_j[x] \boxdot A_f[x],$$

costing $O(N)$ pseudorandom-generator calls for the full-domain evaluation plus $O(N/\ell)$ packed ciphertext-plaintext multiplications and $O(\log \ell)$ rotations, *independent of N_d*. No mask correction is needed on this path, because A_f is derived from the unmasked I held by the DO and is itself encrypted.

The fast path supports the templates registered at setup, namely count, sum, per-sensor-band sums, and fixed plaintext-weighted inner products, applied to a single keyword. Conjunctive queries and ad-hoc weight vectors chosen at query time use the flexible path of Section 6.9. Insertion is cheap: a new record carrying τ tags updates A_f at τ addresses in plaintext at the DO, followed by re-encryption of all of the column's packed ciphertexts, for the reason given in Section 6.10; no decryption is involved.

6.6. Packing the response

The straightforward layout spends two ciphertexts on one scalar: the value and its MAC (Section 6.7), each column packed across all ℓ slots and summed over

all of them, so that $\ell - 1$ slots of each response go unused. BFV slots have more structure than that. With ring dimension ℓ they form two rows of $\ell/2$, and the automorphisms behind rotation act within each row [59]. CR-SHE uses that shape directly. The DO packs A_f in row 0 and $\widehat{A}_f$ in row 1, chunked $\ell/2$ tags per ciphertext; the provider multiplies each chunk by its evaluation vector, replicated in both rows, adds the chunks, and rotates and sums within the rows. Slot 0 then carries the value share and slot $\ell/2$ the MAC share, in one ciphertext.

Several templates share that ciphertext. For k templates, k a power of two, each row is cut into k segments of $s = \ell/(2k)$ slots, template r occupying segment r of both rows, with $\lceil N/s \rceil$ chunks. Rotating and summing by $1, 2, \ldots, s/2$ leaves the sum of segment r in its first slot, so slot rs of each row carries template r's value or MAC share. The response is one ciphertext for any $k \le \ell/2$; what grows with k is the number of chunks, $\lceil kN/(\ell/2) \rceil$ plaintext-ciphertext products, while the rotations fall from $\log_2(\ell/2)$ to $\log_2 s$. The worked instance of a MEAN below needs its count and sum in one response; count, sum and sum of squares give a variance in one round.

Finally, the provider drops the response to a single RNS tower by modulus switching before it sends it. At multiplicative depth one, decryption is unaffected, and the ciphertext halves. Against the two-ciphertext layout, which costs $2\lceil N/\ell \rceil$ products and $2\log_2 \ell$ rotations per template, packing halves the provider's homomorphic work (rotations dominate at these parameters) and quarters the downlink, and for four templates cuts the downlink sixteen-fold (Section 10.5). The natural single-server alternative, one server answering the same query over the same encrypted columns, uses this same layout with an encrypted one-hot query in place of the DPF shares; Section 10.5 measures it.

6.7. Verification of the aggregate

Providers run the same contraction against the MAC column $\widehat{\mathrm{CT}}$ (or $\widehat{A}_f$ on the fast path), which sits in the second slot row of the same ciphertext (Section 6.6), so $\widehat{\mathrm{ct}}_j$ travels with ct_j at no extra downlink. The DU decrypts once and aborts unless $\hat{y} = \delta y$. The check is linear, and that fixes precisely what it covers. A provider that perturbs its response by anything it cannot also MAC, such as adding a constant, returning a stale or foreign ciphertext, or corrupting one of the two halves, is detected except with probability $1/p \approx 2^{-36}$ over the unknown δ, which is adequate for catching faults but is not a cryptographic integrity guarantee. Two independent MAC columns would lower this to $1/p^2 \approx 2^{-72}$, sharing the response through a further segment. A provider that applies the *same* linear substitution to both halves passes: replacing its evaluation vector e_j by any e'_j, for instance $k\,e_j$, shifts y by $\sum_x (e'_j - e_j)[x]\, A_f[x]$ and $\hat{y}$ by δ times as much, so the tag still verifies. The provider cannot aim the shift, since it knows neither A_f nor α_w, but it can make the output wrong undetectably. The MAC therefore protects against faulty and non-adaptive corruption, not against a provider that deviates on purpose; per-row keys (Remark 2) narrow the gap on the fast path, and Section 10.12 measures all of these cases.

Remark 2 (Per-row MAC keys). Replacing the single δ by per-row keys $\delta_x = F''_{K''}(x)$, storing $\widehat{A}_f[x] = \delta_x A_f[x]$ and checking $\hat{y} = \delta_{\alpha_w} y$, costs no additional ciphertexts or operations. A substitution $e'_j = e_j + d$ then passes only if

$$\textstyle\sum_{x \neq \alpha_w} d[x]\,(\delta_x - \delta_{\alpha_w})\,A_f[x] = 0.$$

For any d that touches a row with $A_f[x] \neq 0$ this is a non-zero linear form in independent uniform keys, which vanishes with probability $1/p$. What remains undetected is a deviation concentrated on the queried row itself, which rescales the answer; because the provider's view is independent of α_w, it succeeds with probability at most $\max_x \Pr[\alpha_w = x]$, the predictability of the query. The reference implementation uses per-row keys on the fast path by default, and Section 10.12 measures them against the single key. They do not carry over to the flexible path, whose contraction runs over records rather than tags; there the single δ remains.

6.8. Record retrieval and query authorisation

When the DU needs payloads rather than an aggregate, providers return b_j directly. The DU recovers $b + P[\alpha_w]$, subtracts $P[\alpha_w]$, which it can derive locally from K', and obtains b; no correction ciphertext is involved. This response is $O(N_d \log p)$ bits of *fixed* size, independent of $|S|$, so the coalition learns nothing from its length. The DU then issues a batch of $k_{\max}$ DPF-based payload retrievals padded to a public bound, as in Splinter [12], so the coalition learns only $k_{\max}$.

Authorisation interacts with obliviousness in a way that is easy to overstate. DPF privacy hides α_w from every provider, so no provider can check *which* keyword a query targets: a client permitted any query can read any row, and N honest-format queries recover I', hence I for a client that holds K'. Keys that do not encode a point function add a second capability, that of arbitrary linear combinations of rows or of A_f, such as a single aggregate over the union of many keywords, and cannot be rejected locally either: under the n-party instantiation of Section 3 a key *is* an additive share vector, so no forgery is needed, and a well-formed two-party key is pseudorandom by the privacy property. For a fully trusted DU, which holds sk and the dictionary, none of this exceeds its authority. It matters as soon as authority is partial, for a DU entitled to some keywords or templates and not others, and the remedy is a proof, carried with the key, that it encodes a point

function at a permitted address, as in PACL [18], or a verifiable DPF [22] for well-formedness alone. In the constructions we are aware of, including the sketching checks of client-robust aggregation [17], the servers compare a short verification message, so adopting one relaxes CR-SHE's no-interaction property to a single short exchange per query; whether that exchange can be avoided is open. The flexible path's second round is the same situation in a starker form: a share vector is arbitrary by construction, so round two is an oracle for any linear function of the record column, which a fully trusted DU may use and a partially trusted one must not. Our prototype checks key shape only, and Section 10.13 measures what unrestricted query access extracts. We state per-user authorisation as a requirement rather than a solved problem.

6.9. The flexible path: conjunctions and ad-hoc weights

Registered single-keyword templates cover the common case. For anything else, such as a conjunction or disjunction of keywords, a weight vector chosen at query time, or a threshold on the matched set, the DU runs two rounds.

Round one is a retrieval (Section 6.8) for each keyword involved, after which the DU holds the posting lists and forms whatever selection vector it needs over the records, for example $v = w \circ (b_{w_1} \wedge b_{w_2})$ for an ad-hoc weight vector w and a conjunction of two keywords.

Round two deals v as fresh additive shares $v = \sum_j v_j$ over $\mathbb{Z}_p$, compressed exactly like the n-party key of Section 3: $n-1$ providers receive a λ-bit seed whose PRF expansion is their share, and one receives the explicit difference vector. Each provider runs the contraction of Section 6.3 against the record column, packed with m_i in row 0 and δm_i in row 1 of $\lceil N_d/(\ell/2) \rceil$ ciphertexts, and returns one ciphertext. The DU adds the n responses, decrypts once and checks $\hat{y} = \delta y$. The magnitude bound of Lemma 1 is the DU's to check, against a public bound on m_i, since it alone knows v.

Round two costs $\lceil N_d/(\ell/2) \rceil$ plaintext-ciphertext products and $\log_2(\ell/2)$ rotations per provider, λ bits of uplink to $n-1$ providers and $N_d \lceil \log p \rceil$ bits to one, and one ciphertext down. Every share a coalition of $n-1$ providers holds is uniform, or pseudorandom for the seeds, independently of v, so round two adds to the coalition's view nothing that depends on the query (Theorem 3). The DU learns S and $|S|$, which $\mathcal{L}^{\mathrm{DU}}$ already contains.

A single round is not possible here in general: a conjunction is the product of two selection vectors, which no linear contraction of shares computes, and an ad-hoc weight vector would need a mask correction the DO cannot precompute (Remark 1).

6.10. Dynamic ingest

IoT streams are append-only. CR-SHE keeps a static tier and a hot buffer of at most B records scanned by the same oblivious contraction; every B insertions the buffer is folded into a rebuilt static tier. Because the coalition's per-query view is independent of the query (Theorem 3), forward and backward privacy hold trivially: there is no query-dependent state for an update to correlate against. Three rules keep the update path itself from leaking.

Rows never move. A new keyword takes the next free row of D. The DO provisions N above the current vocabulary, and free rows are all-zero rows masked like any other, so a new keyword changes no public quantity; only exhausting the headroom, which re-provisions a larger N, is visible. Time-window tags make the vocabulary grow for as long as the deployment runs, at one tag per minute on our primary corpus, so a deployment bounds N by retention, keeping one static tier per epoch (a day is 1,440 minute tags) and querying the epochs a template covers.

A fresh mask key per static tier. A rebuild re-lays the record axis, so reusing K' would reuse mask values at new positions, and the difference of two masked indexes would expose the change in membership. The DO therefore draws a fresh K' for every static tier, re-masks the index under it, and gives authorised DUs the current key.

Whole columns are replaced. An insertion changes A_f only at the τ rows its tags occupy, so re-encrypting only the affected packed ciphertexts would tell the coalition which blocks of ℓ rows those tags fall in. The DO re-encrypts all $\lceil N/(\ell/2) \rceil$ ciphertexts of each packed column per update batch, which is 3 to 9 at our corpus sizes, leaving batch times and the public dimensions as the only dynamic leakage.

A rebuild's cost R grows with the store. On the fast path it is dominated by recomputing the precomputed columns from the postings and re-encrypting $\lceil N/(\ell/2) \rceil$ ciphertexts per template; a deployment that also serves retrieval and the flexible path must rebuild and re-replicate the $\Theta(NN_d)$ masked index. Amortisation of R/B therefore holds within one tier generation, and a long-lived deployment should merge tiers geometrically, as log-structured stores do, to keep the amortised cost logarithmic in the store size. Section 10.10 reports both costs.

6.11. Worked instance

A clinician queries the tag `arrhythmia` with the registered template MEAN. The DU derives α_w and sends one DPF key per provider. Each provider performs a full-domain pass, contracts against the SUM and COUNT columns packed side by side, and returns one ciphertext carrying both values and both MACs. The DU adds the shares, decrypts once, verifies the MACs, and divides. No provider, and no coalition of up to $n-1$, learns the tag queried, which records contributed, how many did, or the mean itself.

Algorithm 1 Outsourcing at the data owner.

procedure $\mathsf{Setup}(1^\lambda, n, t, p)$
  $D \leftarrow \emptyset$; $K', K'' \leftarrow \{0,1\}^\lambda$; $\delta \leftarrow \mathbb{Z}_p^*$
  $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda, L_{\mathrm{HE}}, p, \ell)$
  $pp \leftarrow (n, t, N, N_d, p, \ell, \{w_f\})$
  return $(D, K', \delta, pk, sk, pp)$
end procedure

procedure $\mathsf{BuildIndex}(D, K', DB)$
  $D(w) \leftarrow |D|$ for each new w ▷ rows never move
  $I[D(w)][i] \leftarrow 1$ for each keyword w of record i
  $P[x] \leftarrow$ rejection-sampled $F'_{K'}(x, \cdot)$ uniform on $\mathbb{Z}_p^{N_d}$, for all $x \in [N]$
  $I' \leftarrow I + P \bmod p$
  return I' ▷ replicated to all CP_j
end procedure

procedure $\mathsf{Precompute}(pk, K', DB, \{w_f\})$
  for all registered templates f **do**
    $A_f[x] \leftarrow \sum_i I[x][i]\, w_f[i]\, m_i$
    $\widehat{A}_f[x] \leftarrow \delta_x A_f[x]$, $\delta_x \leftarrow F''_{K''}(x)$
    pack A_f in row 0, $\widehat{A}_f$ in row 1, templates in segments (§6.6); encrypt
  end for
end procedure

procedure $\mathsf{EncData}(pk, DB)$
  pack m_i in row 0, δm_i in row 1; encrypt $\lceil N_d/(\ell/2) \rceil$ ciphertexts
  symmetric-encrypt opaque payloads
end procedure

Algorithm 2 Query, at the data user and at provider CP_j.

procedure $\mathsf{Token}(D, w, f)$ ▷ at the DU
  $\alpha_w \leftarrow D(w)$
  $(\kappa_1, \ldots, \kappa_n) \leftarrow \mathsf{Gen}(1^\lambda, \alpha_w, 1)$ over $\mathbb{Z}_p$
  return $(\kappa_j, f) \to \mathrm{CP}_j$
end procedure

procedure $\mathsf{FastAgg}(\kappa_j, f, A_f, \widehat{A}_f)$ ▷ single keyword
  check the shape of κ_j ▷ §6.8
  $e_j[x] \leftarrow \mathsf{Eval}(j, \kappa_j, x)$, $x \in [N]$ ▷ $O(N)$ PRG
  $\mathrm{ct}_j \leftarrow \boxplus_c E_{j,c} \boxdot \mathrm{A}_c$ ▷ $O(N/\ell)$ HE
    $E_{j,c}$: chunk c of e_j in both rows
  rotate and sum within segments; compress
  return ct_j ▷ one ct, value + MAC
end procedure

procedure $\mathsf{Deal}(v)$ ▷ at the DU, flexible
  $v = \sum_j v_j$: seeds to $n-1$ providers, explicit difference to one
end procedure

procedure $\mathsf{FlexAgg}(v_j, \mathrm{CT})$ ▷ round two
  $\mathrm{ct}_j \leftarrow \boxplus_c V_{j,c} \boxdot \mathrm{CT}_c$; rotate and sum
  return ct_j
end procedure

procedure $\mathsf{Retrieve}(\kappa_j, I')$
  return $b_j \leftarrow \sum_x e_j[x] \cdot I'[x]$ ▷ fixed size
end procedure

procedure $\mathsf{CombineDec}(sk, K', \delta, \cdot)$ ▷ at the DU
  $(y, \hat{y}) \leftarrow$ slots $(0, \ell/2)$ of $\mathsf{Dec}(sk, \boxplus_j \mathrm{ct}_j)$
  abort if $\hat{y} \neq \delta_{\alpha_w} y$ (fast) or δy (flexible)
  retrieval: $b \leftarrow (\sum_j b_j) - P[\alpha_w]$
    $S \leftarrow \{i : b[i] = 1\}$
  return y, and S if retrieving
end procedure

7. Security analysis

Leakage profile. We define $\mathcal{L} = (\mathcal{L}_{\mathrm{Setup}}, \mathcal{L}^{\mathrm{CP}}, \mathcal{L}^{\mathrm{DU}})$ with an observer split:

$$\mathcal{L}_{\mathrm{Setup}}(DB) = (n, t, N, N_d, p, \ell, \{w_f\}),$$
$$\mathcal{L}^{\mathrm{CP}}_{\mathrm{Query}} = \mathrm{class}(q),$$
$$\mathcal{L}^{\mathrm{DU}}_{\mathrm{Query}} = \big(|S|,\ f(\mathrm{records}(S))\big).$$

Here $\mathrm{class}(q)$ says whether a query is a fast-path aggregate over a named layout of registered templates, a retrieval, or the second round of a flexible query. The coalition learns that and the public dimensions of the deployment, and nothing else: not the keyword, not the access pattern, not $|S|$, not the weights, not the result. In contrast to a design in which the matched set is returned to the providers for evaluation, and in contrast to an unmasked index, $\mathcal{L}_{\mathrm{Setup}}$ here contains no posting-list lengths and no membership structure, by Theorem 1. Retrieval adds the public padding bound $k_{\max}$ to $\mathcal{L}^{\mathrm{CP}}$. A conjunctive query is two retrievals and one flexible round, adding only result sizes to $\mathcal{L}^{\mathrm{DU}}$, never to the coalition.

Definition 1 ($\mathcal{L}$-security against t-collusion). In $\mathbf{Real}_{\mathcal{A},\Pi}(1^\lambda)$: (1) $\mathcal{A}$ outputs (DB, C) with $|C| \le t$; (2) the challenger runs Setup, $\mathsf{BuildIndex}$, $\mathsf{Precompute}$, $\mathsf{EncData}$ and gives $\mathcal{A}$ the static view $(I', \mathrm{CT}, \widehat{\mathrm{CT}}, \{A_f, \widehat{A}_f\}_f, pk, pp)$; (3) for $q = 1, \ldots, Q$, $\mathcal{A}$ adaptively issues a query (a keyword with a template layout, a keyword to retrieve, or a selection vector v_q for a flexible round) and receives the coalition's per-query view: its DPF keys $\{\kappa_{q,j}\}_{j \in C}$ or its shares $\{v_{q,j}\}_{j \in C}$, and the responses those providers compute; (4) $\mathcal{A}$ outputs a bit. In $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\mathcal{L}}(1^\lambda)$ a PPT simulator $\mathcal{S}$ produces the views given only $\mathcal{L}_{\mathrm{Setup}}$ and the per-query leakage. Π is $\mathcal{L}$-adaptively-secure against t-collusion if for every PPT $\mathcal{A}$ corrupting $\le t$ providers there is a PPT $\mathcal{S}$ with $|\Pr[\mathbf{Real} = 1] - \Pr[\mathbf{Ideal} = 1]| \le \mathsf{negl}(\lambda)$.

Theorem 2 (Correctness). *Fix a database DB, a keyword w, and a registered template f satisfying the magnitude bound of Lemma 1. Then* $\mathsf{CombineDec}$ *returns $y = \sum_{i \in S} w_f[i]\, m_i$ with $S = \{i : I[\alpha_w][i] = 1\}$ on the fast path,* $\mathsf{Retrieve}$ *returns exactly S, and a flexible round on a selection vector v satisfying the magnitude bound returns $\sum_i v[i]\, m_i$, except with the negligible probability that leveled-HE decryption fails.*

Proof. By DPF correctness $\sum_j e_j[x] = 1$ at $x = \alpha_w$ and 0 elsewhere.

Fast path. $\boxplus_j \mathrm{ct}_j$ decrypts, by Lemma 1 applied with $\beta = \big(\sum_j e_j\big)$ over the array A_f, to $A_f[\alpha_w] = \sum_i I[\alpha_w][i] w_f[i] m_i = \sum_{i \in S} w_f[i] m_i$.

Retrieval. $\sum_j b_j = \sum_x \big(\sum_j e_j[x]\big) I'[x] = I'[\alpha_w] = b + P[\alpha_w]$, so $\big(\sum_j b_j\big) - P[\alpha_w] = b$ exactly, and the DU derives $P[\alpha_w]$ from K'.

Flexible round. $\sum_j v_j = v$, so by Lemma 1 applied with $\beta = v$ over the record column, $\boxplus_j \mathrm{ct}_j$ decrypts to $\sum_i v[i]\, m_i$ in slot 0 and $\delta \sum_i v[i]\, m_i$ in slot $\ell/2$.

Packing. Rotations act within slot rows and the rotate-and-sum covers one segment, so the slot that carries template r receives only the products of template r's segment, and the value and MAC rows do not mix.

The dictionary is injective by construction, so row α_w is exactly the records containing w. Decryption correctness under the noise growth discussed in the proof of Lemma 1 contributes the negligible failure term. The MAC check passes with certainty for honest providers, since δ, or δ_{α_w} on the fast path, distributes over the linear contraction: $\sum_x e[x]\, \delta_x A_f[x] = \delta_{\alpha_w} A_f[\alpha_w]$ when $e = u_{\alpha_w}$. □

Theorem 3 (t-collusion leakage security). *If F' is a secure PRF, $(\mathsf{Gen}, \mathsf{Eval})$ is an $(n-1)$-private DPF, and the homomorphic and symmetric schemes are IND-CPA, then CR-SHE is $\mathcal{L}$-adaptively-secure against t-collusion in the sense of Definition 1, for $t = n-1$ and $\mathcal{L}^{CP}_{\mathrm{Query}} = \mathrm{class}(q)$. Concretely, for every* PPT *$\mathcal{A}$ there exist* PPT *$\mathcal{S}$ and $B_1, \ldots, B_4$ with*

$$\big|\Pr[\mathbf{Real}{=}1] - \Pr[\mathbf{Ideal}{=}1]\big| \le \mathrm{Adv}^{\mathrm{PRF}}_{B_1} + Q \cdot \big(\mathrm{Adv}^{\mathrm{DPF}}_{B_2} + \mathrm{Adv}^{\mathrm{PRF}}_{B_4}\big) + m \cdot \mathrm{Adv}^{\mathrm{CPA}}_{B_3},$$

where m is the number of ciphertexts in the coalition view. As is standard for BFV, IND-CPA is assumed to hold even given the relinearisation and rotation keys the providers need (circular security). The theorem is stated for one static tier; with a fresh K' per tier (Section 6.10) it applies to each tier independently, with one PRF term per tier.

Proof. Take $|C| = n-1$ without loss of generality; a smaller coalition holds a subset of the same state. Let $H_0 = \mathbf{Real}$.

H_1 (PRF to random, index to zero). Replace F' by a truly random function, at cost $\mathrm{Adv}^{\mathrm{PRF}}_{B_1}$. The mask is then an exactly uniform one-time pad (Theorem 1), so building I' from the all-zero index instead of I changes nothing; I' now depends only on (N, N_d).

H_2 (DPF keys to simulated). Replace $\{\kappa_{q,j}\}_{j \in C}$ by $\mathcal{S}_{\mathrm{DPF}}(1^\lambda, C, N)$, licensed by $(n-1)$-privacy; a hybrid over the Q queries costs $Q \cdot \mathrm{Adv}^{\mathrm{DPF}}_{B_2}$. The per-query key material now depends only on (C, N).

H_2' (flexible shares to uniform). In each flexible round at least one share lies outside C. If it is the explicit difference vector, the coalition's shares are seed expansions independent of v_q. If it is a seed's expansion, the explicit vector is v_q minus a PRF output the coalition cannot compute, and replacing that output by a uniform vector costs $\mathrm{Adv}^{\mathrm{PRF}}_{B_4}$. Either way the coalition's shares become uniform and independent of v_q, over at most Q rounds.

H_3 (ciphertexts to encryptions of 0). Replace each of the m ciphertexts in the coalition view, namely the record column and the columns $A_f, \widehat{A}_f$, by encryptions of 0 under pk, at cost $m \cdot \mathrm{Adv}^{\mathrm{CPA}}_{B_3}$, since the coalition holds only pk.

Simulator. The responses $\mathrm{ct}_{q,j}$ need not be simulated separately: each is a deterministic function of $\kappa_{q,j}$ or $v_{q,j}$, the query class, and the provider's stored state, all of which $\mathcal{S}$ possesses in H_3. Compression is a public, deterministic map and changes nothing. $\mathcal{S}$ therefore regenerates the entire coalition view from $\mathcal{L}_{\mathrm{Setup}}$ and $\mathrm{class}(q)$ alone, so $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\mathcal{L}}$ matches H_3; summing the terms proves the theorem. □

Remark 3 (Rounds and leakage). Suppose the client recovers S and then requests $\mathsf{Compute}(pk, \{\mathrm{CT}_i\}_{i \in S}, f)$ from the providers. The coalition's per-query view then contains S itself, which is a function of the queried keyword and the database. No simulator given only data-independent leakage can produce it, so an empty per-query leakage profile is *unattainable* in that design rather than merely loose. The flexible path has the same two rounds, but its second round sends fresh shares of the selection instead of the selection, and H_2' shows those are simulatable. What makes Theorem 3 true is contracting a share at the provider, not the number of rounds.

Remark 4 (Confidentiality and degradation). For two databases of equal public dimensions differing only in contents, Theorem 3 makes each coalition view indistinguishable from a distribution depending only on $\mathcal{L}_{\mathrm{Setup}}$, which is identical for both; the executions differ by at most $m \cdot \mathrm{Adv}^{\mathrm{CPA}}$, with the content dependence isolated in H_3. The same proof applies for any $|C| \le n-1$, since $(n-1)$-privacy holds for every strict subset; a coalition of all n providers is outside the model, and against it the guarantee degrades to IND-CPA confidentiality of the records with the index still masked under F'.

Remark 5 (Beyond semi-honest). Section 6.7 detects a response inconsistent with its MAC except with probability $1/p$. It does not detect a provider that applies the same linear substitution to both halves, that is, a contraction against any vector other than its evaluation share, nor does it guarantee output delivery. Full malicious security through verifiable function secret sharing remains future work, which is why Table 1 records *detect* rather than ✓.

8. Complexity analysis

Table 4 summarises the asymptotic costs. Four entries deserve comment because they correct the intuitive accounting.

Table 4
Asymptotic costs. γ is the per-provider DPF key size, $O(\lambda \log N)$ at $n = 2$; ℓ is the SIMD slot count.

Resource	Per provider	Data user
Index storage	$O(NN_d \log p)$	n/a
Ciphertext storage	$O(N_d/\ell) + O(N/\ell)$ per f	n/a
Query uplink	$O(\gamma)$	$O(n\gamma)$
Flexible uplink	λ, or $N_d\lceil \log p \rceil$ bits	$O(N_d \log p)$
Aggregate downlink	1 ct	n ct
Retrieval downlink	$O(N_d \log p)$, fixed	$O(nN_d \log p)$
Fast path	$O(N)$ PRG, $O(N/\ell)$ HE	1 decrypt
Retrieval / round one	$O(N)$ PRG, $\Theta(NN_d)$ ring	$O(N_d)$
Flexible round two	$O(N_d/\ell)$ HE, $O(\log \ell)$ rot.	1 decrypt
Owner precomputation	postings per f; $O(NN_d)$ mask	n/a
Insertion, τ tags	$O(\tau)$ updates, $O(N/\ell)$ re-enc.	n/a

First, the selection contraction of a retrieval, and hence of the flexible path's first round, is $\Theta(NN_d)$ ring operations, not $\Theta(N)$: a full-domain evaluation produces N share values, each of which multiplies an index *row* of width N_d. Counting only the $O(N)$ pseudorandom-generator calls of the evaluation omits the dominant term. With a small modulus and vectorised arithmetic these are cheap word operations, and the pass is embarrassingly parallel, but the asymptotic is what it is: this is the ORAM and PIR lower-bound price for hiding the access pattern from a coalition without preprocessing [24, 35]; preprocessing PIR avoids it at the cost of client hints that updates invalidate [27].

Second, the fast path of Section 6.5 removes the N_d factor entirely, at $O(N)$ generator calls plus $O(N/\ell)$ packed ciphertext operations, at the cost of restricting queries to single keywords and templates registered at setup.

Third, the aggregate response is one ciphertext, for up to $\ell/2$ templates, and, crucially, *independent of* $|S|$. A response whose size grew with the result set would leak the result-set size to the coalition and contradict $\mathcal{L}^{\mathrm{CP}}$. Retrieval responses are $O(N_d \log p)$ bits of fixed size for the same reason.

Fourth, the single-server alternative of Section 6.6 needs no non-collusion assumption but pays in two places. On the fast path its query is $\lceil N/(\ell/2) \rceil$ fresh ciphertexts instead of an $O(\lambda \log N)$ key, and its contraction uses ciphertext-ciphertext products. On retrieval and the flexible path it must multiply an encrypted N-vector by the $N \times N_d$ index, about NN_d/ℓ plaintext-ciphertext products with diagonal packing [59], where CR-SHE does $\Theta(NN_d)$ word operations. Both are $\Theta(NN_d)$: the gap is the constant ratio of a homomorphic slot operation to a machine-word operation, which Section 10.5 bounds from below. What does not shrink with better kernels is the client side: a $\lambda \log N$-bit key against $\lceil N/(\ell/2) \rceil$ fresh encryptions.

9. Implementation

We implement CR-SHE end to end in C++17, together with the corpus builder and one benchmark script per figure. That prototype implements an earlier configuration of the design: the two-ciphertext response, a single MAC key, and mask values reduced from 64-bit words. Its measurements are reported where the quantity measured does not depend on those choices, and are labelled as such.

Reference implementation. We also release an open reference implementation of the construction (Section 11): a Python package driving OpenFHE's C++ kernels (OpenFHE-Python 1.5.1) at identical parameters, with the AES-MMO DPF evaluated level by level through OpenSSL's AES-NI code. It implements the packed response, compression, per-row MAC keys, rejection-sampled masks, retrieval, the flexible path and baseline (vi), and ships 34 tests, including DPF checks at every point of every domain up to depth 8, the n-party sharing for $n \in \{3, 4, 5\}$, all templates for $n \in \{2, 3, 4\}$, noise across several full-magnitude packed chunks, and byte-identical responses for empty and full posting lists. Its latencies are those of a Python driver and are not comparable with the C++ figures; every comparison drawn from it (Sections 10.5, 10.6 and 10.12) puts all rows through the same implementation on the same machine.

Distributed point function. The two-party DPF is the Boyle-Gilboa-Ishai GGM-tree construction with a fixed-key AES-128 pseudorandom generator in the Matyas-Meyer-Oseas form $G(s) = \mathrm{AES}_k(s) \oplus s$ under three independent fixed keys, giving two child seeds and one conversion block per node. Three code paths are provided, for x86-64 AES-NI, ARMv8 crypto extensions, and a portable software fallback, and every binary reports at start-up which one it took; the measurements below were taken with AES-NI. Full-domain evaluation traverses the tree once, processing nodes in pairs so that four independent AES chains are in flight, and converts leaves four at a time. The implementation is verified exhaustively against the point function at every point of the domain for depths up to 12 and for $n \in \{2, 3, 4, 5\}$.

Homomorphic layer. Leveled BFV through OpenFHE 1.2.3 [60], at plaintext modulus $p = 68{,}724{,}326{,}401$, a 37-bit prime with $p \equiv 1 \bmod 2^{17}$ so that batching is available at every ring dimension up to 65,536, ring dimension 8,192, $\ell = 8{,}192$ SIMD slots, multiplicative depth 1, at the 128-bit classical security level. The one contraction is a plaintext-to-ciphertext product followed by a rotate-and-sum over the batch.

Lemma 1 as a type, not a comment. The DPF output group and the BFV plaintext ring are the same object in the code: a single modulus type is constructed once and consumed by the DPF, the index mask, and the homomorphic backend. The failure mode the lemma

warns about, a DPF over $\mathbb{Z}_{2^k}$ composed with an unrelated plaintext modulus, is therefore unrepresentable rather than merely documented. The magnitude bound is checked against the corpus at setup, and a corpus that would violate it is rejected rather than silently wrapped.

Selection arithmetic. The retrieval contraction accumulates in 128 bits and reduces once per output: each product is below $p^2 < 2^{74}$ and at most $N \le 2^{16}$ of them are summed on either corpus, so the accumulator cannot overflow and the inner loop is a single multiply-add. The pass is parallelised over tag blocks with per-thread accumulators.

Deployment. The implementation separates a deployment bundle along the trust boundary: the public half carries the crypto context, the evaluation keys, pk and the precomputed columns, and the provider process loaded from it holds no secret key and refuses decryption. We exercise this with $n \in \{2, 3, 4\}$ independent provider processes and a client communicating over TCP. We report it as a *single-host* deployment (Section 10.7): the processes are genuinely separate and the protocol is the real one, but they run on one machine over the loopback interface, so the transport component is framing and serialisation only and excludes wide-area latency. Section 10.16 returns to what this omits.

Randomness and keys. Every secret the protocol samples, from DPF root seeds to the seeds of the n-party construction and the owner's K' and δ, is drawn from the operating system's CSPRNG, never from a seeded generator. The benchmark harness is the one exception: it derives the owner's keys from a fixed public seed so that the leakage experiment and the correctness tests reproduce bit for bit, and such keys are never used in a deployment bundle. In the C++ prototype, mask values are 64-bit AES-CTR words reduced modulo p, the variant whose statistical term Section 6.4 removes; the reference implementation rejection-samples them. The dictionary assigns rows in sorted tag-name order when a corpus is built; it does not implement the insertion of new keywords or the per-tier re-keying of Section 6.10, which the static benchmarks below do not exercise.

Corpora. The primary corpus is the Environmental Sensor Telemetry dataset of 405,184 MQTT messages from three Raspberry-Pi sensor arrays, each carrying a device identifier, a timestamp, and readings for carbon monoxide, humidity, liquid petroleum gas, smoke, temperature, light and motion [61]. Records are indexed under the metadata tags a deployed IoT platform would attach: the device identifier, a one-minute time window, decile bands of each of the five continuous sensor channels, and the two boolean flags. This yields $N = 11{,}580$ tags over 3,646,656 postings, with heavy-tailed posting lists of median 35, mean 314.9 and maximum 404,702 (Fig. 8). Aggregation operates on the real temperature readings, scaled to hundredths of a degree, so one coherent dataset drives both retrieval and analytics. The largest admissible single-tag aggregate is 908,575,470, leaving 75.6× headroom below p.

That corpus has only three physical devices, so the whole evaluation is repeated on a second one, the UCI Beijing Multi-Site Air-Quality dataset of 420,768 hourly records from *twelve* monitoring sites over four years [62], which triples the tag domain and supplies the device diversity the first corpus lacks. Section 10.14 reports it in full; both corpora are built by the same script, from the published raw files, with no hand curation.

Baselines. We compare against (i) a plaintext linear scan; (ii) a single-server searchable-encryption scheme with PRF tokens and AES-CTR posting lists, which leaks the search pattern, the access pattern and $|S|$; (iii) non-recursive Path ORAM with bucket size $Z = 4$ and a client-side position map, charging one access per matched record plus one for the index entry; (iv) a distributed-trust, search-only baseline in the style of DORY [13], which is our own selection path stopped at the selection vector and therefore isolates exactly the cost of adding oblivious analytics to oblivious selection; and (v) a *faithful* homomorphic equality scan, which pays the cost of a homomorphic equality per record, $\mathrm{eq}(a, b) = 1 - (a - b)^{q-1}$ by Fermat over a plaintext prime q, at the multiplicative depth that implies; a cheaper proxy that charged one homomorphic addition per item would understate this alternative by orders of magnitude. For (v) one full batch of ℓ slots is measured and the corpus total is obtained by multiplying by the batch count; both the measured batch cost and the extrapolation factor are reported, and extrapolated quantities are marked as such throughout. Finally, (vi) a *single server holding the same encrypted columns*, queried with an encrypted one-hot vector in the packed layout of Section 6.6 and answering with ciphertext-ciphertext products. It needs no non-collusion assumption, so it measures what that assumption buys. Baseline (vi) is measured in the reference implementation, against CR-SHE in the same implementation.

Hardware. All measurements were taken on one AMD EPYC 7J13 machine with 64 cores and 64 GB of RAM running Ubuntu 24.04, with AES-NI, GCC 13 and OpenMP. Latencies are medians of 11 runs unless stated otherwise, with the interquartile range recorded per run and drawn as error bars in the figures. Every binary prints the CPU, thread count, AES backend and homomorphic parameters it ran under, and every results row repeats them, so no number in this section can be separated from the machine that produced it. The reference-implementation experiments ran on a cloud VM with four vCPUs of an Intel Xeon at 2.1 GHz with AES-NI and 16 GB of RAM, on one OpenMP thread, with medians of 21 runs and the interquartile

range recorded; their JSON outputs carry the same environment stamp.

What a fast-path latency includes. The protocol always computes both contractions, value and MAC, so every fast-path figure includes the MAC and every aggregate response is two ciphertexts; we did not measure a MAC-free variant. The fast-path figures differ in two respects only: whether the timed region includes the provider's full-domain DPF evaluation, and how many threads that evaluation uses, which matters because its OpenMP team competes with OpenFHE's own thread pool. With the evaluation on all 64 threads (Tables 6, 7 and 14) a query costs 38.8–41.2 ms across the two corpora; with it on one thread, as the deployed provider runs it, 34.3–35.5 ms on the primary corpus (Section 10.11, Table 11); and the two contractions alone, without the evaluation, 33.6 ms. Serialisation is excluded everywhere except the transport column of Table 11. All of these C++ figures use the two-ciphertext layout, so they bound the packed layout from above; in the reference implementation packing halves the provider's time (Section 10.5).

10. Results analysis and discussion

10.1. Correctness

Every measurement below is taken against an implementation that first has to compute the right answer, so we report that check first. On a 5,000-record subsample of the primary corpus, for $n \in \{2, 3, 4\}$ and all four registered templates, the decrypted aggregate equals the plaintext ground truth exactly and the MAC verifies for 1,000 randomly drawn tags per template on the fast path and 50 on the general path; retrieval returns exactly the posting list; and the aggregate downlink is byte-identical for matched sets of size 0 and 4,999. The homomorphic test suite separately multiplies full-magnitude $\mathbb{Z}_p$ coefficients across several packed chunks, the general path's worst case for noise. The same battery passes unchanged when the same build is repeated on a second, 16-core host. This is Theorem 2 in practice, including the mask cancellation of Section 6.4, which is the step most likely to fail silently. Section 10.14 repeats the same battery on the second corpus. The 50 general-path checks exercised the single-round design of Remark 1. The reference implementation runs the battery on the paths of this paper, fast path, retrieval and the flexible path under ad-hoc weights, in its unit tests and on a synthetic corpus shaped like the primary one (`bench/correctness.py`): for 200 random tags at each $n \in \{2, 3, 4\}$, all three paths agree exactly with the plaintext ground truth, and the compressed response is byte-identical, 132,081 B, for the shortest and the longest posting list.

10.2. Selection latency and key size

Table 5 and Fig. 3 report the per-provider full-domain evaluation, which is the whole of the selection phase. Selection depends on the tag domain alone, so one table covers both corpora. At the primary corpus's $N = 11{,}580$ it costs 0.336 ms on one thread and 0.177 ms at sixteen; at $N = 10^6$ it costs 45.7 ms on one thread and 18.8 ms at thirty-two. The two-party key is 266 B at that domain, growing to 368 B at $N = 10^6$, consistent with the $O(\lambda \log N)$ bound.

Table 5
Per-provider selection latency and two-party key size against N. Medians of 21 runs.

N	Latency (ms), T threads					Key
	$T = 1$	4	16	32	64	(B)
1,000	0.032	0.032	0.032	0.032	0.032	198
10,000	0.290	0.175	0.156	0.161	0.183	266
11,580	0.336	0.200	0.177	0.181	0.211	266
100,000	4.418	2.487	1.909	1.928	2.420	317
10^6	45.741	25.701	19.711	18.769	24.401	368

Selection does not scale past about sixteen to thirty-two threads and then regresses: at $N = 11{,}580$ it improves from 0.336 ms to 0.177 ms at sixteen threads but returns to 0.211 ms at sixty-four, and at $N = 10^6$ from 45.7 ms to 18.8 ms at thirty-two and back to 24.4 ms at sixty-four. The per-level work at these domain sizes is too small to amortise thread dispatch across all cores of a 64-core part. A deployment should pin the selection pass to the knee.

These are sub-millisecond quantities on a shared cloud host, and they carry correspondingly loose error bars: repeated runs of the identical binary differ by up to 25% at $N \approx 10^4$, while the $N = 10^6$ figures, where each measurement is tens of milliseconds, repeat to within 1%. The table gives medians of 21 runs, and the interquartile range is recorded for every cell. Conclusions in this subsection are drawn only at the order-of-magnitude level, which is robust to that variation; the like-for-like comparison that matters is made in Section 10.3, where the quantities are tens of milliseconds.

Set against the homomorphic half of the same query, the scale is what matters: selection is under 0.2 ms while the packed contraction that follows it is tens of milliseconds (Section 10.3). The symmetric-key half of CR-SHE is not the bottleneck, which is the design's intent: symmetric work on the selection path, homomorphic operations only on the aggregate.

10.3. Oblivious aggregation

This is the central claim of the paper, and Table 6 and Fig. 5 test it directly.

On the fast path the provider's aggregate latency is 38.8–39.2 ms across a 405× range of record counts, from $N_d = 10^3$ to the full $N_d = 405{,}184$, a spread of ±0.5%, that is, flat. An initial sweep taken in increasing N_d showed the first four points near 53 ms and the last

four near 38.5 ms; because that step is *anti*-correlated with N_d, we repeated the sweep in decreasing order, which reproduced 38.8–39.2 ms at every point including $N_d = 10^3$. The step was therefore a warm-up artefact of run ordering and not a dependence on N_d, and Table 6 quotes the order-controlled run.

The single-round general path of Remark 1, which the flexible path replaces, performed the same $\Theta(NN_d)$ pass as a retrieval followed by a contraction against the record and correction columns. We keep its measurements because that pass is also the flexible path's first round and dominates it; the second round is measured in Section 10.6. On that path the same aggregate costs 103.7 ms at $N_d = 10^3$ and 1,378.2 ms at the full corpus, which is 35.5× the fast path. The growth is mild up to $N_d \approx 5 \times 10^4$, where the fixed homomorphic cost still dominates, and then tracks the $\Theta(NN_d)$ contraction, giving 196.8, 539.5 and 1,378.2 ms at 10^5, 2×10^5 and 4.05×10^5: a factor of 2.74 for the doubling and 2.55 for the 2.03× step that follows. The slight superlinearity is memory bandwidth: at full scale the masked index is 37.5 GB and every query streams all of it.

That storage is the real constraint on retrieval and the flexible path. The masked index is $N \times N_d$ elements of $\mathbb{Z}_p$, which is 9.26 GB at $N_d = 10^5$ and 37.5 GB at the full corpus, *per provider* and replicated n times. It fits on a 64 GB machine and not on a smaller one, and it must be resident for every query. This is the price of a fully general oblivious selection and it is the reason the fast path exists.

Per-query downlink in the two-ciphertext layout is 525,950 B per provider, being two ciphertexts, the aggregate and its MAC; packed and compressed it is 132,081 B for up to four templates (Section 10.5). Either way it is byte-identical for every N_d and, more importantly, for every $|S|$. We verify this directly by querying a tag whose posting list is empty and one that matches 4,999 of the 5,000 records in the test subsample, and comparing the serialised responses byte for byte. A design that returned the selection vector instead would send $O(N_d \log p)$ bits, 800,000 B at $N_d = 10^5$, and would leak $|S|$ through the response size itself.

10.4. Comparison against baselines

Table 7 places CR-SHE against the five baselines on the primary corpus, the same machine and the same probe tag, whose posting list has that corpus's median length $|S| = 35$.

The ordering is the expected one and the magnitudes are the point. A plaintext scan is free and private against nobody. Single-server searchable encryption is also essentially free and leaks the search pattern, the access pattern and $|S|$; its downlink grows with the result set. Path ORAM hides the access pattern from one server at 1.6 μs per access, but pays per matched record, so its cost and its 373 kB downlink

Table 6
Aggregate latency and storage, fast path against the single-round general path, N = 11,580, template SUM, C++ prototype, two-ciphertext layout. Medians of 11 runs (fast) and 3 (general).

N_d	Fast (ms)	General (ms)	I' / provider (GB)	Downlink (B)
10^3	38.8	103.7	0.09	525,950
5×10^3	39.0	106.9	0.46	525,950
10^4	38.8	111.9	0.93	525,950
2×10^4	38.8	121.7	1.85	525,950
5×10^4	39.1	120.7	4.63	525,950
10^5	39.2	196.8	9.26	525,950
2×10^5	39.1	539.5	18.53	525,950
4.05×10^5	38.8	1,378.2	37.54	525,950

both scale with $|S|$ while CR-SHE's do not. The DORY-style distributed-trust baseline, our own selection path stopped at the selection vector, costs 130.4 ms, and two comparisons against it answer different questions. Like for like, the single-round general path computed the same selection and then aggregated obliviously over it in 196.8 ms at the same N_d (Table 6): oblivious analytics added about half again to oblivious selection, not an order of magnitude. The flexible path's first round is this baseline exactly, and its second round is a fixed homomorphic contraction (Section 10.6). The fast path answers the narrower question of a registered single-keyword aggregate in 39.2 ms, which is 3.3× *less* than materialising the selection vector, because it never touches the $N \times N_d$ index; that gain is bought by restricting the query class, not by a cheaper selection.

The faithful homomorphic equality scan is the comparison that depends most on charging the right operation. At $q = 65{,}537$ a single digit suffices for the tag domain, but the Fermat exponentiation needs 16 squarings, plus one level for the product with the record value: multiplicative depth 17, and ring dimension 32,768. One batch of 32,768 slots costs 1.86 s and the corpus needs 13 batches, giving 24.2 s per query, extrapolated, which is 617× the CR-SHE fast path. Charging one homomorphic addition per item instead would place this baseline in the tens of milliseconds and invert the conclusion. The scan is the right comparison for ad-hoc predicates on a single server, but it is not the strongest single-server alternative for a registered aggregate; baseline (vi) is, and Section 10.5 measures it.

10.5. Response packing, and what the trust split buys

Table 8 puts the two-ciphertext layout, the packed layout of Section 6.6, its compressed form and four-template responses through the reference implementation at both corpora's tag domains, together with baseline (vi), a single server holding the same encrypted

Table 7
Baselines, same corpus and machine, $N_d = 10^5$, probe tag with the median $|S| = 35$. † marks an extrapolated value.

System	Server (ms)	Downlink (B)	Leakage to the server(s)
Plaintext scan	< 0.001	8	everything
SSE, 1 server	< 0.001	560	search and access pattern, $\lvert S\rvert$
Path ORAM	0.059†	373,248	$\lvert S\rvert$, via access count
DORY-style search	130.4	800,000	none; no analytics
FHE equality scan	24,157†	n/a	none; 1 server
CR-SHE, fast	**39.2**	**525,950**	class(q) only

Table 8
Fast path per provider: two-ciphertext against packed layouts, and a single server on the same columns (baseline vi). Reference implementation, one thread; medians of 21 runs. k is the number of templates answered.

Layout	k	Provider (ms)	Downlink (B)
$N = 11{,}580$ *tags*			
Two ct (unpacked)	1	53.0	526,022
Packed, 1 ct	1	23.5	263,011
Packed + compressed	1	27.0	132,081
4× two ct	4	194.9	2,104,088
Packed, 1 ct	4	35.4	263,011
Packed + compressed	4	32.7	132,081
Single server (vi)	1	42.6	263,211
$N = 35{,}162$ *tags*			
Two ct (unpacked)	1	55.8	526,022
Packed, 1 ct	1	31.4	263,011
Packed + compressed	1	36.4	132,081
4× two ct	4	245.6	2,104,088
Packed, 1 ct	4	74.2	263,011
Packed + compressed	4	92.3	132,081
Single server (vi)	1	92.5	263,211

columns. All rows come from one implementation on one machine and one thread, so they compare with each other and not with the C++ figures. Byte counts are a property of the parameters and of OpenFHE's serialisation, not of the machine; the C++ prototype's 262,975 B per ciphertext against 263,011 B here is a serialisation-version difference.

Packing halves the provider's work and the downlink. One rotate-and-sum over half-rows replaces two over full rows, so a single template falls from 53.0 to 23.5 ms at $N = 11{,}580$ and from 55.8 to 31.4 ms at $N = 35{,}162$, and the response falls from two ciphertexts to one, 263,011 B. Compressing to one RNS tower halves it again, to 132,081 B, for about 0.5 ms of modulus switching, which is within the run-to-run spread of this shared machine. Four templates in one response cost 35.4 ms and 132,081 B compressed at $N = 11{,}580$, against 194.9 ms and 2,104,088 B for four two-ciphertext responses: 16× less downlink. At $N = 35{,}162$ the four templates need 35 chunks instead of 12, so their cost grows to 74–92 ms while the response stays at one ciphertext; the products grow with kN and the rotations fall with $\log_2(\ell/2k)$.

Table 9
Per-query uplink and client cost: CR-SHE's two-party key against the single server's encrypted query. Reference implementation, one thread.

	Key (B)	Enc. query (B)	Cts	Encrypt (ms)
$N = 11{,}580$	260	789,033	3	14.6
$N = 35{,}162$	294	2,367,099	9	67.5

Against a single server, the trust split buys the client side. Baseline (vi) needs no non-collusion assumption. Its server costs 42.6 ms at $N = 11{,}580$ and 92.5 ms at $N = 35{,}162$, against 23.5 and 31.4 ms for a packed CR-SHE provider, 1.8 and 2.9 times more, because its contraction multiplies ciphertext by ciphertext. Its response is the same one ciphertext. Its query is not (Table 9): 3 and 9 fresh ciphertexts, 789,033 and 2,367,099 B, against a 260 or 294 B DPF key, which is 3,035 and 8,051 times more uplink, and 14.6 and 67.5 ms of client encryption against 0.54 and 0.62 ms of key generation. Query compression by ciphertext expansion [30] would bring the uplink down to one ciphertext at the cost of more server automorphisms; we did not implement it.

On retrieval the single server would have to bring every entry of the $N \times N_d$ index into contact with an encrypted query, at least $\lceil NN_d/\ell \rceil$ plaintext-ciphertext products before any rotation. At the measured 0.092 ms per product that lower bound is 2.6 s at $N_d = 2 \times 10^4$, against 0.76 s for CR-SHE's retrieval in the same implementation, whose word arithmetic is unvectorised; across our sweep the ratio is at least 2.6–3.4×. The separation on every path is therefore a constant factor in server work, set by the cost of a homomorphic slot operation (11 ns here) against a word operation, plus three orders of magnitude in uplink. That is the honest case for the non-collusion assumption: it is worth making where the client is thin or the link is metered, which is the IoT setting, and not otherwise.

10.6. The flexible path

Table 10 reports the two rounds of a flexible query at $N = 11{,}580$ in the reference implementation, swept in decreasing N_d to control for warm-up. Round one, the retrieval, grows linearly with N_d as its $\Theta(NN_d)$ contraction predicts. Round two costs 20.1–35.6 ms with no trend in N_d over this range, because one to five packed products are dwarfed by the twelve rotations of the row sum, and it returns one 132,081 B ciphertext whatever the selection. Its uplink is a 16 B seed to $n-1$ providers and $8N_d$ B to one. Every flexible query in the reference implementation's tests, including

Table 10
Flexible path, $N = 11{,}580$, $n = 2$, reference implementation, one thread. Round-two uplink is the heavy provider's; the others receive 16 B.

N_d	Round 1 (ms)	Round 2 (ms)	Up (B)	Down (B)	I' (GB)
1,000	41.2	20.1	8,000	132,081	0.09
5,000	247.3	35.6	40,000	132,081	0.46
10,000	399.4	21.9	80,000	132,081	0.93
20,000	758.0	26.4	160,000	132,081	1.85

Table 11
Single-host deployment over loopback, medians of 100 queries. Transport excludes wide-area latency.

n	End-to-end (ms)	Provider (ms)	Transport (ms)	Client (ms)
2	44.12	34.82	8.30	1.37
3	45.40	34.85	9.09	1.52
4	52.53	35.48	15.48	1.63

conjunctions, disjunctions and ad-hoc weights, decrypts to the plaintext ground truth.

10.7. Multi-cloud deployment

We ran the full protocol between independent provider processes and a client, in the deployment split described in Section 9, for $n \in \{2, 3, 4\}$, 100 queries each.

Provider compute is flat in n (Table 11): 34.82, 34.85 and 35.48 ms for $n = 2, 3, 4$. This is the measurement that supports the central structural claim about cost, namely that the client waits on the slowest provider and not on their sum. End-to-end latency rises only from 44.1 ms to 52.5 ms across the same range, and the increase is transport, at 8.3, 9.1 and 15.5 ms, because the client receives n responses of 525,950 B each and deserialises them. Client-side combine, MAC verification and decryption cost 1.37–1.63 ms and are essentially independent of everything else, which is what makes the design viable for a thin data user.

The caveat is the one stated in Section 9: all providers ran on one machine over loopback, so the transport figure is framing, serialisation and the loopback stack, and contains no wide-area latency. A real deployment adds the inter-region round-trip to the transport component and leaves provider compute and client cost unchanged. We do not extrapolate a wide-area number from this measurement.

10.8. Communication

At $N = 11{,}580$ the two-party construction sends a 266 B key to each provider, 532 B of uplink in total. The non-succinct n-party instantiation used for $n > 2$ sends 32 B to each of $n - 1$ providers and 92,656 B to one, so total uplink is 92,688 B at $n = 2$, 92,720 B at $n = 3$ and 92,752 B at $n = 4$. Tolerating more collusion is therefore nearly free in this instantiation, but moving beyond two providers at all costs a factor of 174 in uplink over the succinct two-party tree. Fig. 4 plots both against N. A succinct $(n-1)$-private instantiation would reduce the heavy key from $O(N \log p)$ to $O(2^{n/2}\sqrt{N}\lambda)$.

Downlink is 525,950 B per provider for an aggregate in the two-ciphertext layout and 132,081 B packed and compressed, for up to four templates in our configuration, and a fixed $N_d\lceil \log p \rceil$ bits for a retrieval, which is 800,000 B at $N_d = 10^5$, none depending on $|S|$. A flexible round adds one more 132,081 B ciphertext.

10.9. Leakage evaluation

We mount a frequency and volume leakage-abuse attack in the style of Islam et al. [9] and Cash et al. [10] against our own index, twice: once against an unmasked replicated index, whose rows expose their Hamming weight and hence every posting-list length, and once against the masked index of Section 6.4, whose row statistics we compute with the real PRF over all $11{,}580 \times 405{,}184$ entries. The adversary's auxiliary knowledge is the tag-frequency distribution of a similar deployment, modelled by binomially thinning the true posting lengths to a sampled fraction of the records, so the attack is not handed the answer.

Aggregated over the whole tag universe (Fig. 6) the unmasked index yields 4.27% re-identification under perfect auxiliary knowledge, against 0.0086% for random guessing, a 500× lift but low in absolute terms. That aggregate is misleading, and Table 12 shows why: 11,523 of the 11,580 tags are one-minute time buckets of near-identical length, which volume leakage genuinely cannot separate. Every other tag class, meaning the device identifiers, the light and motion flags and all five sensor-band families, is re-identified *completely*, at 100%. These are precisely the tags that carry semantic meaning: which device, which sensor condition.

Against the masked index the same attack recovers 0.00% in every class and at every level of auxiliary knowledge. A more demanding comparison is with the standard padding defences rather than with no defence: padding each posting list to the next power of two, and volume-hiding padding of every row to the longest. The reference implementation provides the same attack against both (`bench/bench_leakage.py`); we have not yet run it on the corpora (Section 10.16). The masked row statistics are not distinguishable from uniform over $\mathbb{Z}_p$ by a 64-bin chi-square test ($\chi^2 = 72.27$ on 63 degrees of freedom, $p_{\chi^2} = 0.198$). This is the empirical counterpart of Theorem 1, evidence for it rather than a proof of it. We note that this experiment is deterministic given the corpus and the key seed, and reproduced to the digit on two different machines and microarchitectures. Section 10.14 repeats the attack on a twelve-site corpus with three times the tag domain and obtains the same per-class picture (Table 15), so neither the 100% unmasked recovery nor the 0%

Table 12
Tag re-identification by metadata class, primary corpus, under perfect auxiliary knowledge.

Tag class	Tags	Unmasked	Masked
Device identifier	3	100%	0%
Light flag	2	100%	0%
Motion flag	2	100%	0%
CO band	10	100%	0%
Humidity band	10	100%	0%
LPG band	10	100%	0%
Smoke band	10	100%	0%
Temperature band	10	100%	0%
One-minute window	11,523	3.80%	0%
Random guessing	n/a	0.0086%	

masked recovery is an artefact of this corpus's three devices.

10.10. Setup, masking and ingest costs

Masking costs a factor of 64 in index storage over a plaintext bitmap, which is $\lceil \log p \rceil$ rounded to the stored word width and is exactly what Theorem 1 buys. The mask itself is cheap to generate: with the reduction the C++ prototype uses, one AES call in counter mode yields two $\mathbb{Z}_p$ values, and building the masked index takes 5.6 s at $N_d = 10^5$, bounded by memory rather than by time. Rejection sampling (Section 6.4) keeps about one value per AES call, so expect about twice that. The correction columns U_f of the single-round design, at 171–218 ms per template, are no longer needed. The fast-path column A_f costs 1.2–12 ms, since it touches only the postings, and occupies 2 ciphertexts (0.53 MB) regardless of N_d. A complete provider bundle for the full corpus, comprising context, evaluation keys, pk and all eight precomputed columns, is 12 MB and takes 0.8 s to produce.

For dynamic ingest, insertion into the hot buffer runs at 4.5–14.1 million records per second, since a record carrying τ tags updates A_f at τ addresses in plaintext at the owner with no decryption. A fast-path static-tier rebuild, recomputing A_f from the postings and re-encrypting its $\lceil N/\ell \rceil = 2$ ciphertexts for one template, costs 14.7–14.9 ms, which amortises to 58.0 μs per insertion at $B = 256$ and 0.91 μs at $B = 16{,}384$, and re-replicates 1.05 MB per template (the column and its MAC column) to each provider. A deployment that also serves retrieval and the flexible path must in addition re-mask the index under the fresh K', at 5.6 s at $N_d = 10^5$ as above, and re-replicate the index itself, which is 9.26 GB per provider at that size; that, not the fast-path rebuild, is the ingest cost of those paths. The online price is the buffer scan, which is the same oblivious contraction over a B-record index and adds 98.8–100.0% to a query, near-independently of B, because the buffer index shares the tag domain and its cost is dominated by the same fixed homomorphic contraction as the static tier. Buffer size therefore trades amortised rebuild cost against a near-doubling of online latency, and the knee is broad.

10.11. Core scaling and throughput

Per-query latency is 34.3, 34.4 and 38.4 ms at 1, 8 and 64 threads: a single fast-path query does not speed up with cores, and degrades slightly once the thread count exceeds the work available. This is not a defect in the selection code, which does scale (Table 5), but a consequence of where the time goes, since the packed homomorphic contraction and its rotate-and-sum dominate, and those are governed by the library's own parallelism.

Throughput does scale, and is the number a deployment cares about: 29.1 queries/s with one client, 56.9 with two, 110.2 with four, and a plateau of 121.6–127.7 from sixteen to sixty-four clients. Median latency rises from 34.4 ms to 516 ms across that range as concurrency saturates the machine, so the useful operating point is around four to sixteen concurrent clients, where throughput is already within 15% of its maximum and latency is still under 135 ms. Fig. 7 plots both.

10.12. Verification, and what it does not cover

In the two-ciphertext layout the MAC column doubles the homomorphic contraction and exactly doubles the downlink, from 262,975 B to 525,950 B. Packed into the second slot row (Section 6.6) it costs neither: the same products and rotations serve both rows. Because the protocol always computes it, every fast-path latency in this section already includes it; timed without the DPF pass, the verified contraction in the two-ciphertext layout costs 33.6 ms (Section 9).

In the C++ prototype we measured two distinct deviations over 200 trials each. A provider that returns an aggregate inconsistent with the MAC ciphertext it also returns is detected in 200/200 trials, as Section 6.7 predicts, the residual $1/p \approx 2^{-36}$ being far below what 200 trials could resolve. A provider that performs a *well-formed* contraction against a different selection vector, consistently in both the value and the MAC column, is detected in 0/200 trials, necessarily, since $\hat{y} = \delta y$ still holds. We report both. The second number is the linearity gap of Section 6.7 made concrete: it is why Table 1 records *detect* rather than a tick, and what per-row MAC keys (Remark 2) or a verifiable DPF would narrow.

Table 13 repeats the experiment in the reference implementation with four deviations, under the single key and under per-row keys, 200 trials each, at $N = 11{,}580$. The single key reproduces the C++ result and extends it: it catches every inconsistent response and none of the consistent substitutions, whether the substituted vector is a uniform perturbation of e_j or a multiple of it. Per-row keys catch all of them. The

Table 13
MAC detection over 200 trials per cell: detected / undetected and wrong / undetected and still correct. $N = 11{,}580$, reference implementation.

Provider deviation	Single δ	Per-row δ_x
Constant on value row	200 / 0 / 0	200 / 0 / 0
$e_j + d$, d uniform	0 / 200 / 0	200 / 0 / 0
$k\,e_j$	0 / 200 / 0	200 / 0 / 0
$e_j + c\,u_x$, random x	0 / 168 / 32	174 / 0 / 26

one deviation they cannot catch is one concentrated on the queried row; perturbing a single random row x was caught in 174 of 200 trials, and in the other 26 the row had $A_f[x] = 0$, so the perturbation changed nothing and the answer was correct. The provider could not aim at α_w, which it does not know. Per-row keys cost nothing at query time and are now the default on the fast path; the flexible path keeps the single key.

10.13. Query authorisation

Section 6.8 argues that providers cannot restrict which rows a client reads. We measure it on a synthetic index of $N = 512$ tags and $N_d = 1{,}024$ records. A client that submits raw additive shares of the basis vectors e_x, which is the n-party key format, so no tree-shaped key is ever produced and the shape check never runs, recovered 512/512 rows and 100% of membership bits with N two-provider queries (1,024 provider evaluations), and neither provider could distinguish the queries from honest traffic, since each share is individually uniform. The cost is linear in N, so the same extraction against the full telemetry index would take 11,580 queries on any deployment that serves retrieval. This is what any client with query access can do against the prototype as it stands, and it is what a per-user authorisation mechanism must prevent.

10.14. Second corpus: twelve devices and a larger tag domain

The primary corpus has three physical devices, so its tag universe is dominated by time buckets and a reader is entitled to ask whether the results depend on that. We therefore repeat the entire evaluation, every correctness test and every experiment above, on the UCI Beijing Multi-Site Air-Quality corpus [62]: 420,768 hourly records from twelve monitoring sites over four years, each carrying six pollutant channels ($PM_{2.5}$, PM_{10}, SO_2, NO_2, CO, O_3) alongside meteorological readings. Records are indexed under the site identifier, the hour, the wind direction, and decile bands of the six pollutant channels together with pressure and dew point. Temperature is the aggregated field; because it goes below zero, it is shifted by $+50\,^{\circ}\mathrm{C}$ before scaling to hundredths of a degree, which keeps every value non-negative as Lemma 1 requires and is removed exactly at the client after a sum is divided by its count. One dataset again drives both retrieval and analytics.

Missing readings are handled explicitly rather than silently. A channel reading `NA` receives its own `:na` tag instead of falling into the lowest quantile band, because CO is missing in 4.92% of records, so band-0 folding would have merged "sensor down" with "clean air", and the 398 records (0.09%) with a missing temperature are dropped, because temperature is the aggregated field and an aggregate must not sum an imputed value. The result is $N_d = 420{,}370$ records and $N = 35{,}162$ tags over 4,624,070 postings, with posting lengths of median 12, mean 131.5 and maximum 102,344: three times the tag domain of the primary corpus, and a much flatter posting distribution (Fig. 2). The largest admissible single-tag aggregate is 698,963,226, leaving $98.3\times$ headroom below p.

Every correctness check passes unchanged: exact agreement with the plaintext ground truth on both paths for $n \in \{2, 3, 4\}$ and all four templates, exact retrieval of the posting list, MAC verification, and a byte-identical 525,950 B downlink for $|S| = 0$ and $|S| = 5000$.

The fast path is unaffected by the larger tag domain. It costs 40.2–41.2 ms from $N_d = 5 \times 10^3$ to $N_d = 4.05 \times 10^5$ (Table 14), against 38.8–39.2 ms on the primary corpus: a 5% increase for $3.04\times$ the tags. The $\lceil N/\ell \rceil$ packed products grow from 2 ciphertexts to 5, but the fixed rotate-and-sum, encryption and serialisation dominate, so the online cost grows far more slowly than N. The first point of this increasing-N_d sweep (54.6 ms at $N_d = 10^3$) carries the warm-up artefact identified in Section 10.3; the order-controlled protocol that removed it on the primary corpus was not re-applied here, so the range above starts at the second point.

The single-round general path, and with it retrieval, degrades exactly as its complexity predicts. At $N_d = 10^5$ it costs 382.0 ms against 196.8 ms on the primary corpus ($1.94\times$ for $3.04\times$ the tags), and its masked index grows linearly in N, from 9.26 to 28.13 GB at that record count. At full scale it would be 118.2 GB per provider. The harness refuses any point whose masked index would exceed 75% of the memory available when it runs, that is, 47.5 of 63.3 GB, the remainder being needed for per-thread accumulators and the encrypted columns, so the last two points, 56.3 and 114.0 GB, are refused, and the refusals recorded. That refusal is the result: on a corpus of this tag cardinality the masked index is storage-bound before it is time-bound, for retrieval and the flexible path alike, which is precisely the argument for the fast path.

The leakage conclusion is not an artefact of three devices. Under perfect auxiliary knowledge the unmasked index gives up *every* one of the ten non-temporal tag classes at 100%, including the twelve site identifiers and the seventeen wind-direction tags, which are device and condition labels in the sense that matters, while the 35,046 hour-bucket tags yield

0.31%. Against the masked index the same attack recovers 0.00% in every class and at every level of auxiliary knowledge, and the masked row sums remain indistinguishable from uniform over $\mathbb{Z}_p$ under the same 64-bin chi-square test ($\chi^2 = 62.38$ on 63 degrees of freedom, $p_{\chi^2} = 0.4986$). Table 15 gives the breakdown: the per-class structure of Table 12 reproduces exactly on a corpus with four times as many devices.

The remaining measurements track the primary corpus with no change of conclusion. Provider compute in the single-host deployment is flat in n (38.67, 37.65, 38.47 ms for $n = 2, 3, 4$), with end-to-end latency rising from 48.17 to 56.04 ms in transport alone. The DORY-style search-only baseline costs 305.6 ms against 41.2 ms for the complete CR-SHE fast path at the same N_d, so the gap *widens* to 7.4× as N grows, because that baseline materialises a selection vector the fast path never touches. The faithful homomorphic equality scan is 24.4 s, essentially unchanged, since it scales with the record count and the Fermat depth rather than with N; that is 592× the fast path. Verification detects 200/200 inconsistent aggregates and 0/200 well-formed contractions against a different selection vector, and the extraction experiment again recovers 512/512 rows with N queries. Throughput plateaus at 117–124 queries/s. Dynamic ingest is likewise unchanged in character: insertion into the hot buffer runs at 4.5–11.1 million records per second, a fast-path static-tier rebuild costs 28.7–29.1 ms, twice the primary corpus and again linear in N, and the buffer scan adds 98.9–99.6% to a query at every buffer size, as before. The one cost that grows materially is setup: building the masked index at $N_d = 10^5$ takes 16.6 s against 5.6 s, linear in N as expected, while the fast-path column A_f remains 5 ciphertexts (1.3 MB) independent of N_d.

Two quantities are deliberately *not* restated for this corpus. Selection latency is a function of the tag domain alone and not of the records, so it is governed by Table 5, whose 10^4 and 10^5 rows bracket $N = 35{,}162$; we do not quote a figure at that domain because our sweep measured the primary corpus's N. Uplink is likewise a property of N, and both keys follow exactly from the constructions: the two-party tree key is 300 B, by the same formula that gives the measured 266 and 317 B, and the n-party heavy key is $8N + 16 = 281{,}312$ B. Both are computed rather than measured. The latter is a further 3× on the factor of 174 discussed in Section 10.8, which strengthens the case for a succinct multi-party instantiation.

10.15. Discussion

The fast path does what it was designed to do. Aggregate latency is flat to ±0.5% over a 405× range of N_d, per-query downlink is one ciphertext independent of $|S|$, 132 kB for four templates, and provider compute is flat in n. A registered single-keyword aggregate costs 3.3× less than a distributed-trust baseline that only selects and 617× less than a faithful homomorphic scan. Selection is linear-work per provider on every path; the fast path removes the dependence on record count but restricts queries to single keywords and templates registered at setup, and the flexible path lifts that restriction for one more round.

What the non-collusion assumption buys is the client side. A single server holding the same encrypted columns answers the fast path with no non-collusion assumption at 1.8–2.9× the server work, and a retrieval at no less than about three times, so on server work alone the assumption buys a constant factor. It buys three orders of magnitude in uplink and one to two in client-side query cost: a 260 B key generated in half a millisecond against megabytes of fresh encryptions costing tens of milliseconds on a server-class core. For an IoT data user on a gateway or a metered link that is the difference that matters, and it is the regime in which we recommend CR-SHE; for a well-provisioned client, a single-server design avoids the assumption at modest cost.

Table 14
Second corpus ($N = 35{,}162$, $N_d = 420{,}370$): aggregate latency and storage, template SUM. Sweep taken in increasing N_d, so the first fast-path point carries the warm-up artefact of Section 10.3. "n/a" marks a point the harness refused on its memory budget; the size it needed is still given.

N_d	Fast (ms)	General (ms)	I' / provider (GB)	Downlink (B)
10^3	54.6	106.3	0.28	525,950
5×10^3	40.4	86.9	1.41	525,950
10^4	40.2	99.7	2.81	525,950
2×10^4	40.5	145.5	5.63	525,950
5×10^4	40.7	220.8	14.06	525,950
10^5	41.2	382.0	28.13	525,950
2×10^5	40.9	n/a	56.26	525,950
4.05×10^5	40.9	n/a	113.98	525,950

Table 15
Tag re-identification by class, second corpus. The per-class structure of Table 12 reproduces on twelve devices.

Tag class	Tags	Unmasked	Masked
Monitoring site	12	100%	0%
Wind direction	17	100%	0%
CO band	11	100%	0%
NO_2 band	11	100%	0%
O_3 band	11	100%	0%
$PM_{2.5}$ band	11	100%	0%
PM_{10} band	11	100%	0%
SO_2 band	10	100%	0%
Pressure band	11	100%	0%
Dew-point band	11	100%	0%
One-hour window	35,046	0.31%	0%
Random guessing	n/a	0.0028%	

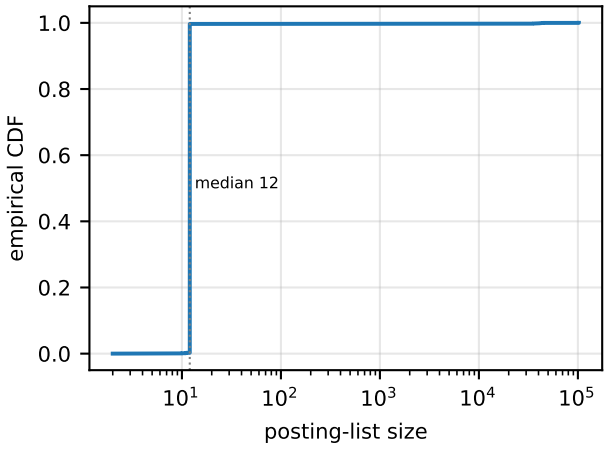


Figure 2: Posting-list size CDF, second corpus. Flatter than the primary corpus (Fig. 8): median 12 against 35.

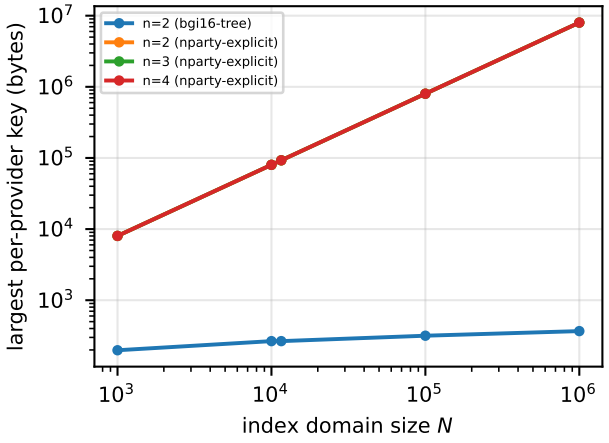


Figure 4: Largest per-provider DPF key against N, for the two-party tree and the non-succinct n-party instantiation.

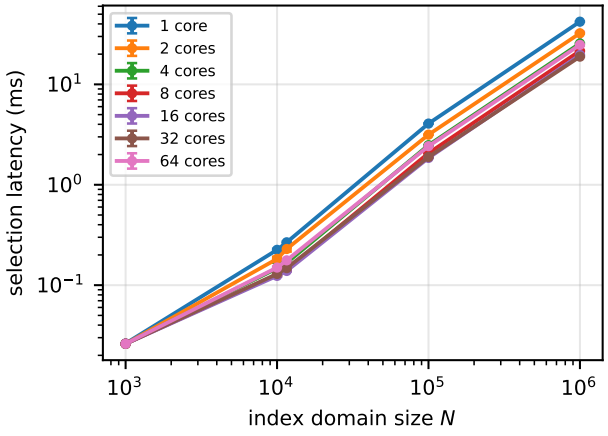


Figure 3: Selection latency against N at several thread counts. Medians of 21 runs; error bars are the interquartile range.

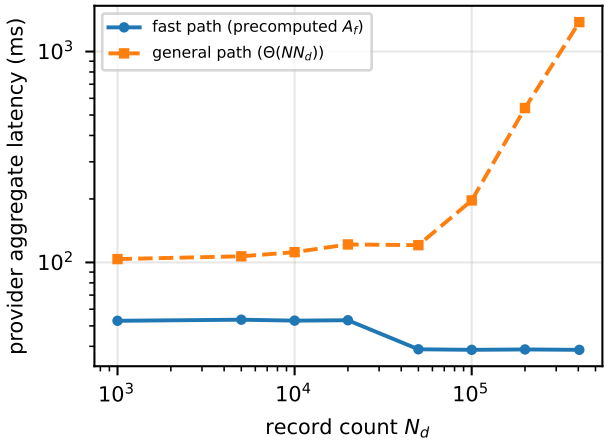


Figure 5: Aggregate latency, fast path against the single-round general path (C++, two-ciphertext layout). The x-axis is N_d, not $|S|$: cost no longer depends on the matched-set size. The fast path is flat to $\pm 0.5\%$ over a $405\times$ range.

Retrieval and the flexible path are bounded by storage, not by time. The $\Theta(NN_d)$ contraction reaches 1.38 s at the full corpus, but the masked index is 37.5 GB per provider on the primary corpus and 118.2 GB on the second, replicated n times and resident for every query. Ad-hoc weight vectors and conjunctions pay that price. Streaming the index from storage in blocks would bound resident memory at the cost of I/O, though not the work, since an aggregate needs every record's contribution; a $\mathbb{Z}_p$ with $p < 2^{32}$ would halve the index at the cost of magnitude headroom.

Device diversity does not change the picture. The primary corpus has three physical devices. The twelve-site corpus, with three times the tag domain, reproduces the per-class leakage result exactly and costs 5% more on the fast path (Section 10.14).

10.16. Limitations

We collect here what the evaluation does not establish.

Deployment and hardware. Providers ran as separate processes on one host, so transport excludes wide-area latency, and no component was measured on gateway-class hardware. The data user's share of the work is one DPF key generation, $2(n-1)$ ciphertext additions and two decryptions, at 1.4–1.6 ms on our server (Table 11), and it is the part that would run at the edge. Measuring it, with energy, on a gateway, and running providers in distinct cloud regions, are the evaluations still to do.

Adversary. Providers are semi-honest. The MAC detects inconsistent responses except with probability

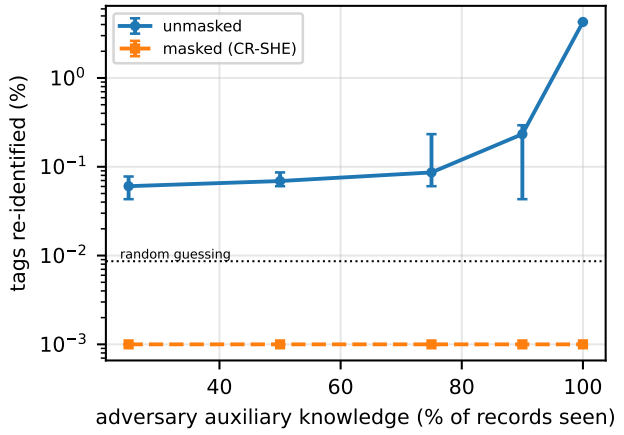


Figure 6: Tag re-identification against the unmasked and masked index, against the adversary's auxiliary knowledge. Table 12 breaks it down by class.

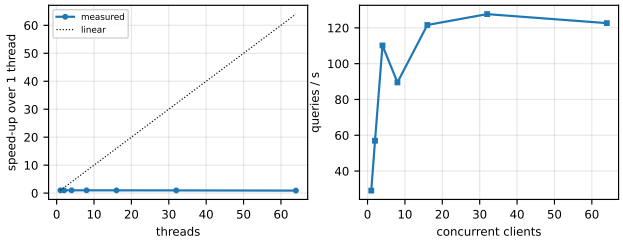


Figure 7: Left: speed-up of one fast-path query with thread count. Right: sustained throughput against concurrent clients.

$1/p$; per-row keys extend that to consistent substitutions on the fast path, except one concentrated on the queried row, but not on the flexible path, and nothing guarantees output delivery or checks a retrieval share (Section 6.7). Per-user query authorisation is a requirement the prototype does not meet (Section 10.13).

Collusion beyond two providers. For $n > 2$ the prototype uses a non-succinct sharing whose heavy key is $8N+16$ bytes, 174× the two-party key on the primary corpus. Our key-size claims are for $n = 2$.

Response size. A packed, compressed aggregate is one 132 kB ciphertext for up to four templates, still large for a handful of scalars. Further modulus reduction below one tower, or a smaller ring for the response, would need a different parameter set, which we have not explored.

Leakage baseline. Our measured attack contrasts the masked index with an unmasked one. The comparison against a padded and a volume-hiding index [36, 37] is implemented but has not been run on the corpora.

Owner and ingest. The owner retains the plaintext and rebuilds each static tier under a fresh mask key. The prototype implements neither the insertion of new

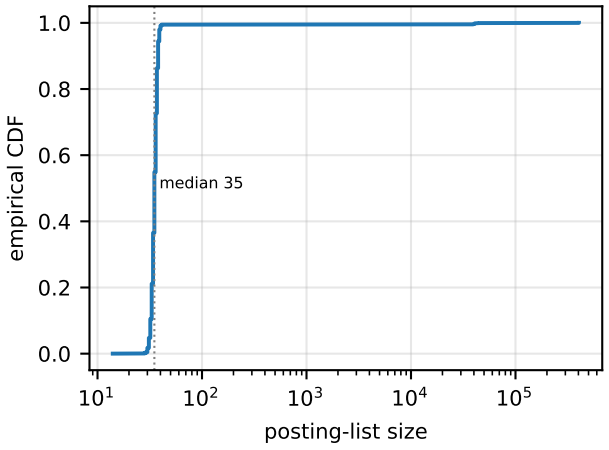


Figure 8: Posting-list size CDF, primary corpus. Retained to characterise the corpus and to set the retrieval padding bound $k_{\max}$.

keywords nor per-tier re-keying (Section 6.10), and its rebuild measurement covers the fast-path columns only.

Model. The construction assumes a single data owner and full delegation of decryption to authorised users; multi-owner sharing and threshold or attribute-based delegation are natural extensions. Post-quantum instantiation follows from the lattice-based homomorphic layer for confidentiality, though the DPF's concrete security must be revisited.

11. Conclusion

This paper addressed a composition leak in distributed-trust query systems: hiding which records a query selects is of limited value if aggregating over them hands the selection back to the servers. CR-SHE removes that leak with oblivious homomorphic aggregation, in which each provider contracts its own distributed-point-function share directly against the homomorphically encrypted record column and returns a single ciphertext, so the matched set is never assembled at any provider and its cardinality never leaves the client. We showed that without server interaction this homomorphic layer cannot be traded for a secret-shared store. A precomputed-column variant makes single-keyword aggregates independent of the record count and returns several of them, with their MACs, in one ciphertext; a two-round flexible path serves conjunctions and ad-hoc weights by dealing fresh shares of the selection; and a one-time-pad index mask removes the posting-list leakage a replicated index otherwise exposes at rest. We proved correctness and adaptive security against a coalition of up to $t = n - 1$ providers with a per-query leakage profile revealing only the requested aggregate class, over data encrypted under a single owner's key and without

any communication between providers. On two real IoT corpora the prototype answers a registered single-keyword aggregate in 39 ms per provider, independently of the record count and of the matched-set size. Measured against a single server holding the same encrypted columns, the trust split's advantage is modest in server work and large at the client, which is where an IoT deployment is constrained. Future work will pursue sublinear preprocessing, verification and authorisation proofs compatible with non-interacting providers, a wide-area multi-cloud and edge-hardware evaluation, multi-owner multi-key deployment, and richer predicates than single-keyword selection.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

Both corpora are public. The Environmental Sensor Telemetry data are available from Kaggle [61] and the Beijing Multi-Site Air-Quality data, under CC BY 4.0, from the UCI Machine Learning Repository [63]; both are rebuilt from the published raw files, with no hand curation, by the loaders in our repository. The reference implementation, the benchmark scripts, the raw JSON outputs of every reference-implementation experiment, and the script that builds Tables 8, 9, 10 and 13 from them are available in a public repository (link withheld for anonymous review; given on the title page).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic), an AI assistant, in order to convert the manuscript between journal templates, edit and restructure the text, write the plotting script that draws Fig. 1, develop the open reference implementation with its tests and benchmark scripts, and draft parts of the analysis in Sections 6.2, 6.9, 6.6, 10.5 and 10.6. No image was generated or altered by a generative AI tool. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] L. Atzori, A. Iera, G. Morabito, The Internet of Things: A survey, Computer Networks 54 (2010) 2787–2805.

[2] D. Petcu, Multi-cloud: Expectations and current approaches, in: Proc. Int. Workshop on Multi-Cloud Applications and Federated Clouds (MultiCloud), 2013, pp. 1–6.

[3] N. Grozev, R. Buyya, Inter-cloud architectures and application brokering: Taxonomy and survey, Software: Practice and Experience 44 (2014) 369–390.

[4] D. X. Song, D. Wagner, A. Perrig, Practical techniques for searches on encrypted data, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2000, pp. 44–55.

[5] R. Curtmola, J. Garay, S. Kamara, R. Ostrovsky, Searchable symmetric encryption: Improved definitions and efficient constructions, in: Proc. 13th ACM Conference on Computer and Communications Security (CCS), 2006, pp. 79–88.

[6] M. Islam, R. Palit, A keyword based searching and sharing scheme on the encrypted cloud data, in: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), IEEE, 2023, pp. 1–6.

[7] J. Fan, F. Vercauteren, Somewhat practical fully homomorphic encryption, Cryptology ePrint Archive (2012).

[8] J. H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, Cryptology ePrint Archive, Paper 2016/421, 2016. URL: https://eprint.iacr.org/2016/421.

[9] M. S. Islam, M. Kuzu, M. Kantarcioglu, Access pattern disclosure on searchable encryption: Ramification, attack and mitigation, in: Proc. Network and Distributed System Security Symposium (NDSS), 2012.

[10] D. Cash, P. Grubbs, J. Perry, T. Ristenpart, Leakage-abuse attacks against searchable encryption, in: Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS), 2015, pp. 668–679.

[11] P. Grubbs, K. Sekniqi, V. Bindschaedler, M. Naveed, T. Ristenpart, Leakage-abuse attacks against order-revealing encryption, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2017, pp. 655–672.

[12] F. Wang, C. Yun, S. Goldwasser, V. Vaikuntanathan, M. Zaharia, Splinter: Practical private queries on public data, in: Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 299–313.

[13] E. Dauterman, E. Feng, E. Luo, R. A. Popa, I. Stoica, DORY: An encrypted search system with distributed trust, in: Proc. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2020, pp. 1101–1119.

[14] E. Dauterman, M. Rathee, R. A. Popa, I. Stoica, Waldo: A private time-series database from function secret sharing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2022.

[15] S. Eskandarian, H. Corrigan-Gibbs, M. Zaharia, D. Boneh, Express: Lowering the cost of metadata-hiding communication with cryptographic privacy, in: Proc. 30th USENIX Security Symposium, 2021.

[16] H. Corrigan-Gibbs, D. Boneh, Prio: Private, robust, and scalable computation of aggregate statistics, in: Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 259–282.

[17] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, Y. Ishai, Lightweight techniques for private heavy hitters, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2021, pp. 762–776.

[18] S. Servan-Schreiber, S. Beyzerov, E. Yablon, H. Park, Private access control for function secret sharing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2023.

[19] N. Gilboa, Y. Ishai, Distributed point functions and their applications, in: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2014, pp. 640–658.

[20] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing, in: Advances in Cryptology – EUROCRYPT 2015, volume 9057 of *LNCS*, Springer, 2015, pp. 337–367.

[21] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing: Improvements and extensions, in: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security, 2016, pp. 1292–1303.

[22] L. de Castro, A. Polychroniadou, Lightweight, maliciously secure verifiable function secret sharing, in: Advances in Cryptology – EUROCRYPT 2022, volume 13275 of *LNCS*, Springer, 2022.

[23] B. Chor, E. Kushilevitz, O. Goldreich, M. Sudan, Private information retrieval, Journal of the ACM 45 (1998) 965–981.

[24] A. Beimel, Y. Ishai, T. Malkin, Reducing the servers' computation in private information retrieval: PIR with preprocessing, in: Advances in Cryptology – CRYPTO 2000, volume 1880 of *LNCS*, Springer, 2000, pp. 55–73.

[25] K. Storrier, A. Vadapalli, A. Lyons, R. Henry, Grotto: Screaming fast $(2+1)$-PC for $\mathbb{Z}_{2^n}$ via $(2,2)$-DPFs, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023.

[26] J. Doerner, a. shelat, Scaling ORAM for secure computation, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017.

[27] H. Corrigan-Gibbs, D. Kogan, Private information retrieval with sublinear online time, in: Advances in Cryptology – EUROCRYPT 2020, LNCS, Springer, 2020.

[28] H. Corrigan-Gibbs, A. Henzinger, D. Kogan, Single-server private information retrieval with sublinear amortized time, in: Advances in Cryptology – EUROCRYPT 2022, LNCS, Springer, 2022.

[29] M. Zhou, A. Park, W. Zheng, E. Shi, Piano: Extremely simple, single-server PIR with sublinear server computation, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2024.

[30] S. Angel, H. Chen, K. Laine, S. Setty, PIR with compressed queries and amortized query processing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2018.

[31] M. H. Mughees, H. Chen, L. Ren, OnionPIR: Response efficient single-server PIR, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2021.

[32] S. J. Menon, D. J. Wu, Spiral: Fast, high-rate single-server PIR via FHE composition, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2022.

[33] A. Davidson, G. Pestana, S. Celi, FrodoPIR: Simple, scalable, single-server private information retrieval, Proceedings on Privacy Enhancing Technologies 2023 (2023).

[34] E. Kushilevitz, R. Ostrovsky, Replication is not needed: Single database, computationally-private information retrieval, in: Proceedings of the 38th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 1997, pp. 364–373.

[35] O. Goldreich, R. Ostrovsky, Software protection and simulation on oblivious rams, Journal of the ACM (JACM) 43 (1996) 431–473.

[36] S. Kamara, T. Moataz, Computationally volume-hiding structured encryption, in: Advances in Cryptology – EUROCRYPT 2019, LNCS, Springer, 2019.

[37] S. Patel, G. Persiano, K. Yeo, M. Yung, Mitigating leakage in secure cloud-hosted data structures: Volume-hiding for multi-maps via hashing, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019.

[38] W. Xu, J. Zhang, Y. Yuan, Z. Li, Privacy-preserving multi-cloud based dynamic symmetric searchable encryption, in: 2021 2nd International Conference on Computer Communication and Network Security (CCNS), IEEE, 2021, pp. 176–181.

[39] W. Xu, J. Zhang, Y. Yuan, X. Wang, Symmetric searchable encryption with supporting search pattern and access pattern protection in multi-cloud, Concurrency and Computation: Practice and Experience 35 (2023) e7651.

[40] X. Zhang, C. Huang, Y. Su, J. Qin, Secure, dynamic, and efficient keyword search with flexible merging for cloud storage, IEEE Transactions on Services Computing 17 (2024) 2822–2835.

[41] N. Yang, C. Tang, Q. Zhou, D. He, Dynamic consensus committee-based for secure data sharing with authorized multi-receiver searchable encryption, IEEE Transactions on Information Forensics and Security 18 (2023) 5186–5199.

[42] R. Du, H. Jiang, M. Li, Lightweight searchable encryption with small clients on edge cloud, in: 2022 17th Asia Joint Conference on Information Security (AsiaJCIS), 2022, pp. 41–48. doi:10.1109/AsiaJCIS57030.2022.00017.

[43] S. Halder, T. Newe, Enabling secure time-series data sharing via homomorphic encryption in cloud-assisted iiot, Future Generation Computer Systems 133 (2022).

[44] A. Kamble, M. M. Jiet, C. Puri, Homomorphic encryption and its applications in multi-cloud security, in: 2024 International Conference on Inventive Computation Technologies (ICICT), IEEE, 2024, pp. 1493–1499.

[45] L. Burkhalter, A. Hithnawi, A. Viand, H. Shafagh, S. Ratnasamy, TimeCrypt: Encrypted data stream processing at scale with cryptographic access control, in: Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2020, pp. 835–850.

[46] L. Burkhalter, N. Küchler, A. Viand, H. Shafagh, A. Hithnawi, Zeph: Cryptographic enforcement of end-to-end data privacy, in: Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 387–404.

[47] C. Xu, N. Wang, L. Zhu, K. Sharif, C. Zhang, Achieving searchable and privacy-preserving data sharing for cloud-assisted e-healthcare system, IEEE Internet of Things Journal 6 (2019) 8345–8356.

[48] D. Wu, C. Nie, Z. Yang, P. Zhang, R. Wang, Distributed secure data sharing method for internet of vehicles based on homomorphic hash tree, IEEE Transactions on Vehicular Technology 73 (2024) 17572–17585.

[49] A. K. Verma, P. Patel, V. S. Bhatia, A. Rastogi, A. Prasad, P. Ranjan, Secure federated multi-party computation with homomorphic encryption and blockchain for preserving privacy in cloud storage of medical records, in: 2025 2nd International Conference on Intelligent Algorithms for Computational Intelligence Systems (IACIS), IEEE, 2025, pp. 1–6.

[50] N. Bharot, N. Mehta, J. G. Breslin, P. Verma, Cloudlock: secure data sharing using a hybrid cryptosystem in multi-cloud data storage, Cluster Computing 28 (2025) 464.

[51] S. Muthuvel, S. Priya, K. Kumar, Design of a multi-layered privacy-preserving architecture for secure medical data exchange in cloud environments, Scientific Reports 16 (2026).

[52] L. Wang, Z. Yang, X. Song, Shamc: A secure and highly available database system in multi-cloud environment, Future Generation Computer Systems 105 (2020) 873–883.

[53] D. Horkos, L. Perret, SoK: On cryptography for multi-cloud storage, Cryptology ePrint Archive, Paper 2026/207, 2026. URL: https://eprint.iacr.org/2026/207.

[54] R. Poddar, T. Boelter, R. A. Popa, Arx: An encrypted database using semantically secure encryption, Proceedings of the VLDB Endowment 12 (2019).

[55] N. Crooks, M. Burke, E. Cecchetti, S. Harel, R. Agarwal, L. Alvisi, Obladi: Oblivious serializable transactions in the cloud, in: Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018.

[56] E. Dauterman, V. Fang, I. Demertzis, N. Crooks, R. A. Popa, Snoopy: Surpassing the scalability bottleneck in oblivious storage, in: Proc. 28th ACM Symposium on Operating Systems Principles (SOSP), 2021.

[57] O. Goldreich, S. Goldwasser, S. Micali, How to construct random functions, in: providing sound foundations for cryptography: On the Work of Shafi Goldwasser and Silvio Micali, 2019, pp. 241–264.

[58] G. Di Crescenzo, T. Malkin, R. Ostrovsky, Single database private information retrieval implies oblivious transfer, in: Advances in Cryptology – EUROCRYPT 2000, volume 1807 of *Lecture Notes in Computer Science*, Springer, 2000, pp. 122–138.

[59] S. Halevi, V. Shoup, Algorithms in HElib, in: Advances in Cryptology – CRYPTO 2014, volume 8616 of *Lecture Notes in Computer Science*, Springer, 2014, pp. 554–571.

[60] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, et al., OpenFHE: Open-source fully homomorphic encryption library, in: Proc. 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC), 2022, pp. 53–63.

[61] G. A. Stafford, [dataset] environmental sensor telemetry data, Kaggle Datasets, 2020. Version 1.0. https://www.kaggle.com/datasets/garystafford/environmental-sensor-data-132k.

[62] S. Zhang, B. Guo, A. Dong, J. He, Z. Xu, S. X. Chen, Cautionary tales on air-quality improvement in Beijing, Proceedings of the Royal Society A 473 (2017) 20170457. Dataset: UCI Machine Learning Repository, Beijing Multi-Site Air-Quality Data, https://archive.ics.uci.edu/dataset/501.

[63] S. Chen, [dataset] Beijing multi-site air quality, UCI Machine Learning Repository, 2019. doi:10.24432/C5RK5G, https://doi.org/10.24432/C5RK5G.